

Palestinian Arabizi and Arabic Transliteration

This paper studies Palestinian Arabizi-Arabic transliteration as a character-level sequence-to-sequence problem. The project compares a shared bidirectional Transformer, a dedicated Arabizi-to-Arabic model, direct Arabic-to-Arabizi baselines, and a two-stage Arabic-to-Arabizi pipeline that first predicts Arabic harakat/diacritics and then generates Arabizi. Results show that the dedicated Arabizi-to-Arabic model outperforms the shared bidirectional baseline, reaching 75.8 percent test sequence accuracy and 91.7 percent top-10 coverage. For Arabic-to-Arabizi, preserving five custom harakat hypotheses before Romanization gives the strongest practical result, reaching 64.1 percent test sequence accuracy and 98.3 percent top-10 coverage. The results support a pipeline design for the harder Arabic-to-Arabizi direction because ordinary Arabic omits pronunciation cues that Arabizi often represents.

I. Introduction

Arabizi is informal Arabic written with Latin letters and numerals. It is common in chat, social media, and situations where Arabic keyboard entry is inconvenient. The writing style is practical for users but difficult for software because it is not a strict one-to-one transliteration scheme. A single Latin character may correspond to several Arabic letters, digits may stand for Arabic sounds with no common Latin equivalent, vowels may be omitted or over-specified, and spelling conventions vary by dialect and by writer. These properties make Arabizi transliteration a sequence prediction problem rather than a simple lookup table.

The two directions are not equally difficult. Arabizi-to-Arabic begins with a user spelling that already contains many pronunciation choices: Latin vowels, doubled letters, and numerals often reveal what the writer intended. Arabic-to-Arabizi begins from ordinary Arabic script, which usually leaves short vowels and other harakat unwritten. That means the model must first infer pronunciation information that is not present in the characters. A human reader would use sentence context, morphology, dialect knowledge, and common sense to decide what the missing harakat should be. In this project, the model often receives only a word or short phrase, so that inference is the core challenge.

The goal of this project is to build a proof-of-concept bidirectional transliteration model set for Palestinian Arabic. The system is intentionally small enough to study and run locally, but it still uses modern neural sequence modeling. For Arabizi-to-Arabic, one model maps Latin-script input to undiacritized Arabic. For Arabic-to-Arabizi, the system first predicts possible diacritized Arabic forms and then uses those candidate forms as structured input to the Arabizi generator. The important contribution is the modeling pipeline and its evaluation, not a finished user application.

The design follows the observation that Arabic script often omits short vowels while Arabizi often expresses them. Direct Arabic-to-Arabizi mapping from unvocalized Arabic can therefore be under-specified. A bare Arabic consonantal form may have several plausible pronunciations, and each pronunciation may lead to more than one Romanized spelling. Even after pronunciation is inferred, there is usually no single correct Arabizi output because writers choose between spellings such as short or doubled vowels, numerals or letter combinations, and English-like or Arabic-sound-like conventions. The project handles this ambiguity by generating up to five harakat hypotheses before generating Arabizi candidates. This choice mirrors the practical behavior of a human reader: first infer likely vocalization, then render the result in a Latin-script convention.

II. Related Work

The core modeling approach is based on neural sequence-to-sequence learning. Sutskever, Vinyals, and Le showed that an encoder-decoder neural network could map variable-length input sequences to variable-length output sequences and achieve strong machine-translation results [1]. Bahdanau, Cho, and Bengio then introduced attention as a way for the decoder to focus on relevant source positions during generation [2]. The Transformer later replaced recurrent computation with multi-head attention and feed-forward layers, improving parallelism and becoming the standard architecture for many sequence transduction tasks [3]. This project uses character-level Transformer encoder-decoder models because transliteration is primarily about spelling and symbol sequences, and because the available data is word-like rather than sentence-scale.

Arabic diacritization is also central to this system. Arabic text is commonly written without short-vowel marks, even though those marks carry phonological, morphological, and sometimes semantic information [6]. Neural diacritization research has shown that character and sequence models can restore these marks effectively when enough clean supervision is available [7]. In this project, diacritization is not the final user-facing output in the Arabic-to-Arabizi direction. Instead, it is an intermediate representation that supplies possible pronunciations to the Romanization model.

The implementation uses PyTorch for training and model serialization. PyTorch is designed around imperative Python execution while still supporting hardware acceleration, which makes it well suited for experimental deep-learning workflows [8]. The dataset foundation is Maknuune, a large open Palestinian Arabic lexicon with more than 36,000 entries that include diacritized Arabic orthography and phonological information [9].

III. Linguistic Background

A key linguistic issue in this project is the difference between Arabic script with and without diacritics. Arabic diacritics, often called harakat, are small marks written above or below letters. They can indicate short vowels, vowel absence, consonant doubling, and other pronunciation information. For example, the consonantal skeleton of a word may stay the same while the harakat change how it is

pronounced. In fully vocalized writing, these marks make pronunciation more explicit.

When diacritics are not included, a lot of information about pronunciation and sometimes meaning can be lost. The same undiacritized Arabic letter sequence can correspond to more than one vocalized word. A reader may infer the intended form from context, but a machine model seeing only a short isolated word has less context than a human reader. This is especially important for transliteration into Arabizi because Arabizi often writes vowels with Latin letters. If the Arabic input does not show the short vowels, the model must guess which vowels should appear in the Arabizi output.

This is why the Arabic-to-Arabizi side of the project uses a two-stage design. The first model predicts possible harakat forms from the plain Arabic input. The second model uses up to five of those vocalized hypotheses to generate Arabizi. The harakat stage is critical as it addresses a real missing-information problem. Without some estimate of the unwritten vowels and pronunciation marks, the Romanized output would be forced to invent vowel information from a stripped Arabic source.

The transliteration goal makes this mismatch especially costly. The harakat adder is not being used to make formal Arabic look complete; it is being used to infer pronunciation before generating Arabizi. If an outside MSA-oriented diacritizer inserts standard-Arabic vowels, the next model may produce Arabizi that reflects the wrong dialectal pronunciation. Training a project-specific harakat adder on the Maknuune-derived Palestinian data keeps the intermediate vocalization closer to the same dialectal distribution as the Arabizi targets. In other words, the custom harakat adder is a domain-adaptation step. It makes the Arabic-to-Arabizi pipeline internally consistent with colloquial Palestinian Arabic rather than forcing the pipeline through an MSA pronunciation layer.

IV. Dataset and Preprocessing

The starting point is the cleaned Maknuune-derived file ``maknuune-v1.0.1_cleaned.csv``, which contains 36,302 rows and three primary fields: ``arabizi``, ``arabic_harakat``, and ``arabic_stripped``. Maknuune was used because it was the strongest available Palestinian Arabic resource found for this project: it is dialect-specific, includes Arabizi, includes diacritized Arabic forms, and is large enough to support supervised neural training. The dataset is still narrower and cleaner than real intelligence text, but for a Palestinian-focused transliteration prototype it is a better fit than a larger MSA corpus or a broad Arabic resource that does not preserve the Arabizi/harakat relationship needed here.

The ``arabizi`` field is the Latin-script representation used as the source for Arabizi-to-Arabic training and as the target for Arabic-to-Arabizi training. The ``arabic_harakat`` field preserves Arabic script with diacritics. The ``arabic_stripped`` field removes those diacritics and is used as the plain Arabic target or source depending on direction. This compact schema supports the prototype model chain without requiring separate annotation passes.

The source-target construction is direction-specific. For Arabizi-to-Arabic, the source is ``arabizi`` and the target is ``arabic_stripped``. For the harakat adder, the source is ``arabic_stripped`` and the target is ``arabic_harakat``. For Levanti-harakat-to-Arabizi, the source is ``levanti_harakat_prediction`` and the target is ``arabizi``. For harakat-top-one-to-Arabizi, the source is ``harakat_top_prediction`` and the target is ``arabizi``. For harakat-top-five-to-Arabizi, the source is the ordered list of up to five harakat predictions, with blanks allowed when fewer than five exist, and the target is ``arabizi``. The top-five structure lets the prototype model learn from uncertainty in the preceding diacritization stage instead of pretending that only one vocalized form is possible.

The data split uses 29,041 training rows, 3,630 validation rows, and 3,631 test rows for the harakat adder, with analogous splits in the transliteration experiments. Character vocabularies are learned from the training columns, and special tokens handle padding, sequence start, and sequence end. Because this is a course project focused on words and short phrases, character modeling is a practical choice. It preserves digits such as 2, 3, and 7 in Arabizi and preserves Arabic letters and combining marks in the Arabic side without requiring a large word vocabulary.

Artifact	Rows	Main columns	Purpose
<code>maknuune-v1.0.1_cleaned.csv</code>	36,302	<code>arabizi; arabic_harakat; arabic_stripped</code>	Base paired lexicon
<code>maknuune-v1.0.1_cleaned_predictions_top_5.csv</code>	36,302	base columns plus <code>harakat_prediction_1..5</code>	Reusable input for Arabic-to-Arabizi stage two
<code>maknuune-v1.0.1_cleaned_levanti_harakat_predictions.csv</code>	36,302	base columns plus <code>levanti_harakat_prediction</code>	Levanti-diacritizer comparison input
<code>arabizi_to_arabic_best.pt</code>	--	model state; vocabularies; config; metrics	Best Arabizi-to-Arabic checkpoint
<code>harkat_adder.pt</code>	--	model state; vocabularies; config; best metrics	Arabic diacritization checkpoint
<code>train_arabic_w_harakat_to_arabizi_best--.pt</code>	--	model state; vocabularies; config; metrics	Best gold-harakat-to-Arabizi checkpoint
<code>train_arabic_wo_harakat_to_arabizi_best.pt</code>	--	model state; vocabularies; config; metrics	Best no-harakat-to-Arabizi checkpoint
<code>train_arabic_w_levanti_harakat_to_arabizi_best.pt</code>	--	model state; vocabularies; config; metrics	Best Levanti-harakat-to-Arabizi comparison checkpoint
<code>train_harakat_top_1_to_arabizi_best.pt</code>	--	model state; vocabularies; config; metrics	Best top-one-to-Arabizi comparison checkpoint
<code>harakat_top_5_to_arabizi_best.pt</code>	--	model state; vocabularies; config; metrics	Best top-five-to-Arabizi checkpoint

Table I. Principal data and model artifacts used by the system.

Undiacritized Arabic	Number of Arabizi variants	Example Arabizi spellings
عقد	8	3aqad; 3aqid; 3uqad; 3uqud; 3akkad; 3akkid; 3aqqad; 3aqqid
نقر	7	naqar; naqir; nuqur; naqqar; naqqir; nuqqar; nuqqur
زور	6	zoor; zuur; zavar; ziwir; zawwar; zawwir
فرق	6	farq; faraq; fariq; firaq; farraq; farriq
نفذ	6	nafad; nafaz; naffad; naffaz; naffid; naffiz

Table II. Examples of multiple Arabizi spellings for the same undiacritized Arabic form in the cleaned Maknuune-derived data.

V. Model Development

All transliteration components are character-level encoder-decoder Transformer models. The Arabizi-to-Arabic, original-harakat-to-Arabizi, no-harakat-to-Arabizi, Levanti-harakat-to-Arabizi, harakat-top-one-to-Arabizi, harakat-top-five-to-Arabizi, and shared bidirectional baseline models use a model dimension of 192, eight attention heads, three encoder layers, three decoder layers, a feed-forward dimension of 512, dropout of 0.15, and saved vocabularies for source and target characters. The harakat adder is slightly deeper and wider in the feed-forward block: it uses a model dimension of 192, six attention heads, four encoder layers, four decoder layers, and a feed-forward dimension of 768. This reflects the added complexity of predicting Arabic combining marks.

The deep learning choice is central to the project. Arabizi transliteration has many context-dependent character mappings: `7` may represent 3`, ح` may represent 2`, ع` may represent a hamza-like sound, and Latin vowels can represent short vowels, long vowels, or writer-specific spelling habits. A rule system can encode common substitutions, but it struggles when multiple mappings interact across a whole word. A sequence-to-sequence neural model learns these interactions from paired examples. The encoder converts the entire source character sequence into contextual representations, and the decoder generates the target sequence one character at a time while attending to relevant source positions.

The models operate at character level rather than word level because the vocabulary is small and the task is mostly orthographic. Word-level modeling would create a sparse vocabulary and would fail on unseen spellings. Character-level modeling allows the system to generalize to spelling variants that were not seen as exact tokens during training. It also lets the model preserve numerals and Arabic combining marks as first-class symbols. This is especially important in intelligence or investigative text processing, where informal data often contains rare names, abbreviations, typos, and locally meaningful spellings.

The harakat-adder model trains a diacritizer from plain Arabic to diacritized Arabic. In the Arabic-to-Arabizi direction, this model is used only as an intermediate pronunciation stage. Softmax-90 means the decoder keeps generating alternatives until the retained candidates account for roughly 90 percent of the probability mass or until the configured candidate limit is reached. The goal is not to treat harakat restoration as the final task, but to pass plausible pronunciations forward to the Arabizi generator.

The harakat model works by treating diacritization as character-level sequence generation. Its source string is Arabic with the harakat removed, and its target string is the same lexical item with the expected Palestinian Arabic diacritics restored. During encoding, the model builds contextual representations for the stripped Arabic characters. During decoding, it generates the vocalized output one character at a time, including ordinary Arabic letters and combining marks. This is more difficult than simply inserting vowels into fixed slots because the output sequence can include additional combining characters and because the correct marks depend on dialectal pronunciation patterns learned from the training data.

The harakat-top-five-to-Arabizi model trains the second Arabic-to-Arabizi stage and is the approach selected for the proof-of-concept Arabic-to-Arabizi pipeline. It receives up to five candidate vocalized Arabic strings as the source representation and predicts Arabizi. This is more informative than using only stripped Arabic because Arabizi often contains vowel letters that correspond to short vowels or pronunciation choices. It is also more robust than a single predicted harakat string because the model can learn from several plausible pronunciations instead of depending on one upstream decision. During inference, blank slots are passed when the harakat adder produces fewer than five candidates. The second model then returns up to ten softmax-90 Arabizi outputs stacked in descending likelihood.

The shared bidirectional baseline is included to test the obvious alternative: one Transformer checkpoint for both directions. It uses the same 192-dimensional encoder-decoder architecture as the transliteration models, but prefixes the source sequence with a direction token such as `` or ``. Its Arabizi-to-Arabic examples map `arabizi` to `arabic\_stripped`; its Arabic-to-Arabizi examples map `arabic\_stripped` to `arabizi`. This makes it a fair direct bidirectional baseline, but it does not receive the top-five harakat candidate representation used by the selected two-stage pipeline.

Model	Source	Target	Layers	Heads	Best epoch	Validation seq. acc.	Test seq. acc.
Harakat adder	arabic_stripped	arabic_harakat	4 enc / 4 dec	6	83	0.4983	not stored in bundle
Arabizi to Arabic	arabizi	arabic_stripped	3 enc / 3 dec	8	89	0.7533	0.7580
Original harakat to Arabizi	arabic_harakat	arabizi	3 enc / 3 dec	8	87	0.8280	0.8312
No harakat to Arabizi	arabic_stripped	arabizi	3 enc / 3 dec	8	88	0.4714	0.4738
Levanti harakat to	levanti_harakat_pre	arabizi	3 enc / 3 dec	8	98	0.4932	0.5016

Arabizi	diction					
Harakat top-1 to Arabizi	harakat_top_predict arabizi	3 enc / 3 dec	8	78	0.5138	0.5152
Harakat top-5 to Arabizi	harakat_prediction_arabizi	3 enc / 3 dec	8	98	0.6378	0.6409
Shared bidirectional baseline	direction token + arabic_stripped/arabizi/arabic_strip arabizi ped	3 enc / 3 dec	8	78	0.5725	0.5755

Table III. Model configurations and sequence-accuracy results from saved artifacts.

Bidirectional Baseline

A shared bidirectional baseline was added so the project could directly test the one-model alternative instead of only arguing against it. The baseline is trained from both directions at once and uses a source-side direction token to tell the decoder which script to produce. This is similar to multilingual sequence-to-sequence training: the network shares parameters, while the control token specifies the desired output direction.

This baseline is useful, but it is not equivalent to the selected pipeline. Arabizi-to-Arabic is mostly a normalization problem from an expressive Romanized spelling into Arabic script. Arabic-to-Arabizi is a reconstruction problem followed by generation: the model must infer missing harakat and then choose among several possible Arabizi spellings. Combining both directions into one model forces one training objective to cover two different information conditions.

The completed shared model selected epoch 78 as its best validation checkpoint, with overall validation sequence accuracy 0.5725. On the test set it reached 0.5755 overall sequence accuracy, split into 0.4508 for Arabic-to-Arabizi and 0.7001 for Arabizi-to-Arabic. This confirms that the shared model learned both tasks, but it lagged the specialized direction-specific models.

Shared baseline direction	Test exact seq. acc.	Top-1	Top-3	Top-5	Top-10	Interpretation
Arabic to Arabizi	0.4508	0.4668	0.7059	0.7685	0.7970	Below no-harakat and far below top-five harakat pipeline
Arabizi to Arabic	0.7001	0.7548	0.8689	0.8837	0.8922	Useful, but below the specialized Arabizi-to-Arabic model

Table IV. Shared bidirectional baseline exact-match and softmax-90 top-k results.

Arabic-to-Arabizi experiment	Input evidence	Why it was trained	Main result
Original harakat	Gold `arabic_harakat`	Upper-bound style test when pronunciation cues are visible	0.8312 test exact match
No harakat	`arabic_stripped`	Baseline for ordinary written Arabic without short vowels	0.4738 test exact; 0.874 top-10
Custom top-5 harakat	`harakat_prediction_1..5`	Prototype pipeline preserving pronunciation uncertainty	0.6409 test exact; 0.983 top-10
Custom top-1 harakat	`harakat_top_prediction`	Ablation testing one custom diacritization hypothesis	0.5152 test exact; 0.875 top-10
Levanti harakat	`levanti_harakat_prediction`	External dialect-aware diacritizer comparison	0.5016 test exact; 0.886 top-10

Table V. Arabic-to-Arabizi experiment map showing the role of each trained model.

VI. Training Procedure

Each trainer follows the same broad pattern. First, it reads the cleaned CSV input and validates that the expected source and target columns are available. Second, it builds source and target character vocabularies, encodes strings into indexed tensors, and splits rows into training, validation, and test partitions. Third, it constructs the Transformer model from explicit configuration values and trains with validation checks after each epoch. Finally, it records metrics and plots so the final report can compare models by loss, token accuracy, sequence accuracy, top-k coverage, and length-bucket behavior.

The saved model files include source and target vocabularies, padding indexes, architecture parameters, and selected metrics. This is important because a character-level transliterator is tightly coupled to its vocabulary. If a model were evaluated with a different character ordering, predictions would be meaningless even if the tensor shapes matched. Saving the vocabulary and configuration alongside the weights prevents that class of silent error and makes the reported metrics tied to the exact trained model.

The training recipe was intentionally conventional so that the project would be understandable as a graduate deep-learning prototype. The experiments use seed 42, batch size 128, a maximum of 100 epochs, AdamW optimization with learning rate 3e-4 and weight decay 1e-4, cross-entropy loss with padding ignored, gradient clipping at norm 1.0, and a OneCycle learning-rate schedule. The transliteration models use early stopping patience of 12 validation-loss checks and save checkpoints every five epochs, while the harakat-adder model uses patience 10. These choices keep the training setup clear and make differences between experiments easier to attribute to input representation rather than hidden training changes.

The training metrics also support model selection. The best checkpoint is selected by validation sequence accuracy, not simply by the final epoch. This matters because training loss can continue to decrease while validation exact-match accuracy plateaus or declines. The

Arabizi-to-Arabic model, for example, trained through epoch 100 but selected epoch 89 as the best checkpoint. The Levanti-harakat-to-Arabizi model selected epoch 98. The harakat-top-one-to-Arabizi model selected epoch 78. The harakat-top-five-to-Arabizi model selected epoch 98. The harakat adder selected epoch 83. In all cases, the report uses the saved best-model metrics rather than assuming that the last epoch was best.

VII. Evaluation Methodology

The experiments report token accuracy, sequence accuracy, loss, length-bucket breakdowns, and top-k candidate coverage. Token accuracy counts individual output characters. Sequence accuracy is stricter: the entire predicted output must exactly equal the reference string. In transliteration, sequence accuracy can look harsh because a single character difference, an optional vowel, or a plausible alternate spelling counts as a full error. This issue is especially important for Arabic-to-Arabizi, where there may be several acceptable Romanized spellings for the same Arabic form. For a ranked candidate list, top-k accuracy is often more meaningful because the desired answer only needs to appear among the returned choices.

For practical intelligence use, softmax-90 top-k coverage is the primary operational metric in this report. A watchlist, search expansion tool, or analyst triage interface usually does not require the model's first guess to be the single exact reference spelling. It needs the relevant spelling to be present in the candidate set so it can be searched, reviewed, reranked, or passed downstream. This makes top-k coverage closer to an operational recall measure, while exact sequence accuracy is better understood as a strict diagnostic of one-best model behavior.

Top-k evaluation was intentionally limited to 1,000 samples for the softmax-90 candidate generators. This was done to control processing cost. For each sampled item, the model generates candidates in descending likelihood and records whether the gold reference appears in the top 1, 3, 5, or 10. The same analysis is also broken down by source and target length, allowing the report to separate short-token performance from longer phrase behavior. The softmax-90 threshold is useful because it preserves a compact set of likely alternatives rather than returning every low-probability string the decoder could imagine.

VIII. Results

The main results are easiest to read by separating the two transliteration directions and the intermediate harakat stage. The tables below emphasize top-k coverage because the intended intelligence-style use case is recall-oriented: a useful spelling only needs to survive into a compact candidate list for search expansion or analyst review. Exact sequence accuracy is still shown as a diagnostic, but the top-10 column is the most important practical number.

Arabizi-to-Arabic approach	How it was built	Exact seq. acc.	Top-1	Top-3	Top-5	Top-10	Takeaway
Shared bidirectional model	One Transformer with direction tokens; Arabizi half maps to `arabic_stripped`	0.7001	0.7548	0.8689	0.8837	0.8922	Works, but sharing capacity hurts
Single-direction model	Dedicated Transformer from `arabizi` to `arabic_stripped`	0.7580	0.7750	0.8880	0.9040	0.9170	Best Arabizi-to-Arabic result

Table VI. Arabizi-to-Arabic result summary emphasizing top-k coverage.

Harakat-adding approach	How it was built	Top-1 harakat exact	Top-k/coverage	How it was used
Custom harakat adder	Project-trained character Transformer from `arabic_stripped` to `arabic_harakat`	0.6453 overall prediction exact	0.9499 target in softmax-90 candidates	Generated custom top-1 and top-5 inputs for Arabizi models
Levanti harakat adder	External `guyomorlan/levanti_arabic2diacritics` model applied to stripped Arabic	0.2837 exact against project	not applicable; one external prediction	Generated one Levanti-predicted input for the Arabizi model

Table VII. Harakat-adding result summary. The custom model produced ranked candidates; the Levanti comparison produced one prediction.

Arabic-to-Arabizi approach	Source given to Arabizi model	Exact seq. acc.	Top-1	Top-3	Top-5	Top-10	Takeaway
Original Arabic with harakat	`arabic_harakat`	0.8312	--	--	--	--	Upper-bound style result when gold pronunciation is visible
Original Arabic without harakat	`arabic_stripped`	0.4738	0.4650	0.7510	0.8260	0.8740	Fair baseline for ordinary Arabic input
Levanti-added harakat	`levanti_harakat_pr ediction`	0.5016	0.5060	0.7810	0.8400	0.8860	One external pronunciation hint helps modestly
Custom harakat	`harakat_top_predic	0.5152	0.5050	0.7610	0.8210	0.8750	One custom

top-1	tion`						pronunciation helps exact match but not top-10
Custom harakat top-5	`harakat_prediction_1..5`	0.6409	0.6540	0.9220	0.9670	0.9830	Best practical Arabic-to-Arabizi approach

Table VIII. Arabic-to-Arabizi result summary emphasizing top-k coverage for intelligence-style recall.

Arabizi-to-Arabic Approaches

The Arabizi-to-Arabic direction was tested in two ways: a shared bidirectional model and a single-direction model. Both use character-level Transformer encoder-decoder architecture, but they make different modeling assumptions. The shared model asks whether one checkpoint can learn both script directions at once. The single-direction model gives all capacity and all training signal to the Arabizi-to-Arabic normalization task.

The shared bidirectional model was constructed by doubling the training examples and adding a direction token to the source sequence. An Arabizi input begins with `` and targets `arabic\_stripped`; an Arabic input begins with `` and targets `arabizi`. This is similar to multilingual sequence-to-sequence training, where a control token tells the decoder what language or script to produce. It was included because a reader might reasonably ask why the project did not simply train one model for both directions. The result was useful but weaker than the specialized alternatives: overall test sequence accuracy was 0.5755, with 0.7001 on the Arabizi-to-Arabic half. Its Arabizi-to-Arabic softmax-90 top-k coverage was 0.7548 at top-1, 0.8689 at top-3, 0.8837 at top-5, and 0.8922 at top-10.

The single-direction Arabizi-to-Arabic model was constructed as a direct transliterator from the Maknuune `arabizi` column to `arabic\_stripped`. This model is simpler than the bidirectional baseline because it does not have to share parameters with Arabic-to-Arabizi. That design is justified because Arabizi-to-Arabic is mostly a normalization problem. Latin vowels, doubled letters, and numerals such as 2, 3, and 7 often reveal the user's intended pronunciation. The model completed 100 epochs and selected epoch 89 as the best checkpoint by validation sequence accuracy. It reached 0.7580 test sequence accuracy, with top-k coverage of 0.775 at top-1, 0.888 at top-3, 0.904 at top-5, and 0.917 at top-10. This outperformed the Arabizi-to-Arabic side of the shared bidirectional model, supporting the choice to keep a dedicated reverse-normalization model.

Arabic-to-Arabizi Without Added Harakat

The Arabic-to-Arabizi direction was first tested without any predicted harakat stage. This group contains two important comparisons: a model trained with pre-existing gold harakat and a model trained without harakat. Together they measure how much visible pronunciation information matters.

The model trained with pre-existing harakat uses `arabic\_harakat` as its source and `arabizi` as its target. This is the pronunciation-visible upper-bound condition: it asks how well the Transformer can Romanize Arabic when the short vowels, shadda, sukun, and other marks are already present. The architecture is the same 192-dimensional character-level Transformer used by the other transliteration models. It reached 0.8312 test sequence accuracy, the strongest one-best Arabic-to-Arabizi result in the project. However, this is not the normal user workflow because ordinary users usually type Arabic without full harakat.

The same gold-harakat-trained model was also stress-tested on stripped Arabic input. That test performed badly: top-1 was 0.084, top-3 was 0.168, top-5 was 0.220, and top-10 was 0.309. This result is important because it shows that a model trained to expect harakat cannot simply be handed unvocalized Arabic at inference time. The input representation changed too much.

The model trained without harakat uses `arabic\_stripped` as its source and `arabizi` as its target. This is the clean baseline for ordinary written Arabic. It selected epoch 88 as the best checkpoint and reached 0.4738 test sequence accuracy. Its candidate coverage was much better than its single-best accuracy: top-1 was 0.465, top-3 was 0.751, top-5 was 0.826, and top-10 was 0.874. The model can infer many short and common forms directly, but it lacks explicit pronunciation evidence, so it falls well short of the gold-harakat model and the final top-five pipeline.

Arabic-to-Arabizi With Custom Harakat Added

The next group adds harakat before Arabizi generation. The first stage is the project's own harakat adder, trained from `arabic\_stripped` to `arabic\_harakat` on the same Palestinian Maknuune-derived distribution as the downstream transliteration models. Architecturally, it is a character-level Transformer encoder-decoder like the transliteration models, but slightly deeper and wider in the feed-forward block: model dimension 192, six attention heads, four encoder layers, four decoder layers, feed-forward size 768, and dropout 0.15. This model restores Arabic letters plus combining marks one character at a time.

The harakat adder is not used as a final user-facing model. It is an intermediate pronunciation model. Its best validation checkpoint was epoch 83, with validation sequence accuracy 0.4983 and validation loss 0.2680. It performed best on short-to-medium strings: 10-19 character inputs reached about 0.5710 validation sequence accuracy, while longer inputs dropped sharply. That performance is imperfect, but useful for this pipeline because the downstream model can consume more than one candidate rather than depending on a single forced vocalization.

The custom top-one-to-Arabizi model uses only `harakat\_top\_prediction`, the single most likely output from the custom harakat adder,

as the source for Arabizi generation. It was trained as an ablation: if the top-one model performed nearly as well as top-five, the simpler pipeline might be enough. It selected epoch 78 as the best checkpoint and reached about 0.5152 test sequence accuracy. Its softmax-90 top-k coverage was 0.505 at top-1, 0.761 at top-3, 0.821 at top-5, and 0.875 at top-10. This improved one-best exact match over the no-harakat baseline, but top-10 coverage remained almost the same, showing that one guessed vocalization does not preserve enough uncertainty.

The custom top-five-to-Arabizi model uses up to five harakat hypotheses from the custom adder as the source representation. This is the selected Arabic-to-Arabizi pipeline because it keeps several plausible pronunciations alive before Romanization. It selected epoch 98 as the best checkpoint and reached 0.6409 test sequence accuracy. Its softmax-90 coverage was 0.654 at top-1, 0.922 at top-3, 0.967 at top-5, and 0.983 at top-10. This is the strongest practical result for ordinary Arabic input: it starts from unvocalized Arabic, adds project-specific Palestinian harakat candidates, and returns a high-recall Arabizi candidate list.

Arabic-to-Arabizi With Levanti Harakat Added

The Levanti-harakat approach uses an outside diacritization stage instead of the project's own harakat adder. The external model was ``guymorlan/levanti_arabic2diacritics``, a Hugging Face Levantine/Palestinian-style Arabic diacritizer. Its internal architecture was not trained or modified in this project; it was used as a ready-made dialect-aware harakat predictor. The downstream Arabizi generator was still the same project architecture: a 192-dimensional character-level Transformer encoder-decoder trained from ``levanti_harakat_prediction`` to ``arabizi``.

This approach tests whether a general Levantine pronunciation hint is enough for the final Arabizi task. It is effectively a Levanti top-one path because only one Levanti-predicted vocalized form is passed forward. The downstream model selected epoch 98 as the best checkpoint, with validation sequence accuracy 0.4932. On test data it reached about 0.5016 sequence accuracy. Its top-k coverage was 0.506 at top-1, 0.781 at top-3, 0.840 at top-5, and 0.886 at top-10.

The Levanti path was helpful as an external comparison, but it did not match the custom top-five pipeline. Its one-best exact match was slightly better than the no-harakat baseline and similar to the custom top-one setup, but its top-10 coverage was far below the custom top-five model's 0.983. The most likely explanation is not that Levanti is poor; it is that the final task benefits from a harakat stage trained on the same Maknuune-derived Palestinian data and, more importantly, from preserving multiple pronunciation hypotheses rather than collapsing the upstream uncertainty to one string.

Overall Result Pattern

The results form a readable ladder. For Arabizi-to-Arabic, a dedicated single-direction model works better than the shared bidirectional model because the task is mostly normalization from an expressive Romanized spelling into Arabic script. For Arabic-to-Arabizi, the model with gold harakat proves that the Transformer can Romanize well when pronunciation is visible, while the no-harakat baseline shows the penalty for ordinary unvocalized Arabic. Adding one predicted harakat string helps only modestly, whether that string comes from the custom adder or the Levanti model. Preserving five custom harakat hypotheses is the strongest approach because it gives the Arabizi generator multiple plausible pronunciations before asking it to choose Latin-script spellings.

From the intelligence-workflow perspective, the top-k results are the headline finding. The single-best exact-match numbers show how often the model chooses the reference spelling by itself. The softmax-90 top-k numbers show whether the useful spelling survives into a candidate list for search expansion, analyst review, or downstream NLP. For Arabic-to-Arabizi, where missing harakat and non-standard spelling make one-best prediction inherently uncertain, top-10 coverage is a better practical metric than top-1 exact match alone.

Model	Top-1	Top-3	Top-5	Top-10	Sample size
Shared bidirectional, Arabic to Arabizi	0.467	0.706	0.769	0.797	527
Shared bidirectional, Arabizi to Arabic	0.755	0.869	0.884	0.892	473
Original harakat trained, stripped input stress test	0.775	0.888	0.904	0.917	1,000
No harakat to Arabizi	0.084	0.168	0.220	0.309	1,000
Levanti harakat to Arabizi	0.465	0.751	0.826	0.874	1,000
Harakat top-1 to Arabizi	0.506	0.781	0.840	0.886	1,000
Harakat top-5 to Arabizi	0.505	0.761	0.821	0.875	1,000
	0.654	0.922	0.967	0.983	1,000

Table IX. Softmax-90 top-k coverage for candidate-generating transliteration models.

IX. Error Analysis

The Arabizi-to-Arabic error analysis shows a steep length effect. For source strings of 0-9 characters, sequence accuracy was about 0.8214. For 10-19 characters, it fell to about 0.4215. For 20-29 characters, it was about 0.0313. Target-length buckets show the same broad behavior: 0-9 character targets reached about 0.7996, while 10-19 character targets reached only 0.0739. This does not mean the model has no partial knowledge for longer strings; token accuracy remains high. It means that exact full-string matching becomes statistically fragile as more characters must all be correct at once.

The no-harakat-to-Arabizi baseline shows the penalty for removing pronunciation cues. Source strings of 0-9 characters reached about 0.5015 sequence accuracy, while 10-19 character source strings reached only about 0.0170. Target strings of 0-9 characters reached about 0.5075, and 10-19 character targets reached about 0.3141. The model can often infer common short-word spellings, but phrase-level exact matching deteriorates quickly because each missing vowel or spelling choice compounds.

The harakat-top-five-to-Arabizi model has a different length profile. It performs well on many mid-length examples because the source representation contains multiple pronunciation hypotheses. Source lengths of 10-19 characters reached about 0.7625 sequence accuracy and 20-29 characters reached about 0.5798, while 30-39 characters dropped to about 0.3793. Target-length buckets show similar degradation as outputs become longer. This suggests that the top-five harakat representation gives the model useful disambiguating information, but it cannot fully remove compounding sequence risk.

The harakat-adder errors are especially consequential in the Arabic-to-Arabizi pipeline. If the first stage misses a vocalization, the second stage may never see the pronunciation needed to generate the desired Arabizi output. The top-five strategy reduces this risk but does not eliminate it. Increasing the harakat candidate count would likely improve coverage but also increase compute cost and may introduce noisy candidates. The current system chooses five as a practical compromise between preserving uncertainty and limiting noise.

Direction	Length type	0-9	10-19	20-29	30-39
Arabizi to Arabic	source	0.8214	0.4215	0.0313	--
Arabizi to Arabic	target	0.7996	0.0739	0.0769	0.0000
Original harakat to Arabizi	source	0.9012	0.6689	0.0000	0.0000
Original harakat to Arabizi	target	0.9115	0.4272	0.0000	0.0000
No harakat to Arabizi	source	0.5015	0.0170	0.0000	0.0000
No harakat to Arabizi	target	0.5075	0.3141	0.0000	0.0000
Levanti harakat to Arabizi	source	0.5075	0.5589	0.0000	0.0000
Levanti harakat to Arabizi	target	0.5353	0.3482	0.0000	0.0000
Top-1 harakat to Arabizi	source	0.5167	0.5524	0.0000	0.0000
Top-1 harakat to Arabizi	target	0.5491	0.3547	0.0000	0.0000
Top-5 harakat to Arabizi	source	0.6149	0.7625	0.5798	0.3793
Top-5 harakat to Arabizi	target	0.6290	0.7637	0.5709	0.2949

Table X. Selected sequence-accuracy buckets from length-based error analysis.

X. Threats to Validity and Feasibility Boundaries

Because this is a prototype, the evaluation should be read as evidence of feasibility rather than proof of readiness for operational deployment. The strongest limitation is dataset realism. Maknuune is a clean and valuable Palestinian Arabic lexical resource, and it was the best available dataset found for this project because it directly connects Palestinian Arabic, Arabizi, and diacritized Arabic forms. However, it is not the same as noisy real-world intelligence text. Social-media and messaging-style Arabizi can contain full sentences, emojis, hashtags, usernames, code-switching with English, spelling mistakes, repeated letters, sarcasm, abbreviations, and irregular punctuation. The present system is strongest on words and short phrases that resemble the lexicon-style training examples. A production system would need an external test set drawn from realistic Arabizi data and would need preprocessing for noisy text.

Another limitation is the absence of a formal baseline comparison. A full study should compare the Transformer models against simpler alternatives such as a hand-built character substitution table, finite-state transliteration rules, edit-distance lookup against the lexicon, or a nearest-neighbor retrieval method. Those baselines would answer whether the neural approach provides enough benefit to justify its complexity. This project partially motivates the neural approach by pointing to ambiguous mappings and strong top-k coverage, but it does not yet quantify gains over a simpler baseline.

The model architecture was also not selected through a full hyperparameter search. The chosen Transformer dimensions are reasonable for a small character-level task, but the study does not prove that 192 hidden dimensions, three or four encoder-decoder layers, dropout 0.15, or eight attention heads are optimal. The current architecture should therefore be understood as a practical prototype configuration, not as an optimized model family. A stronger deep-learning study would train smaller and larger variants and report whether performance is limited by model capacity, data size, or decoding strategy.

Exact-match metrics also have limits. Arabizi has many acceptable spellings, and Palestinian Arabic itself can have multiple plausible Arabic-script renderings. A model output can be useful to a human analyst while still failing exact sequence accuracy against a single reference. That limitation matters less for the intended application than it would for a one-shot spelling converter, because the goal is not to produce one official spelling. The goal is to help intelligence users expand Arabic keywords into searchable Arabizi variants for Boolean queries and recall-oriented triage. Top-k metrics address this problem by measuring whether the reference appears among ranked candidates, but they still rely on one gold answer. A stronger evaluation would add character error rate, edit distance, and human judgments of whether non-reference candidates are acceptable.

The harakat stage is a known bottleneck. Its best validation sequence accuracy is about 49.8 percent, and its performance drops sharply on strings longer than about twenty characters. The Arabic-to-Arabizi pipeline can only use the pronunciation hypotheses the harakat adder provides. If the correct vocalization is missing from the top five, the second-stage model may never generate the best Arabizi output. This is a real pipeline-complexity risk, but it is also the main reason the pipeline exists: in the absence of much larger Palestinian Arabic training data that could let a direct model learn around missing harakat, preserving several pronunciation hypotheses

is the most practical way to reduce information loss. It is acceptable for a feasibility prototype focused on short inputs, but it is not sufficient for long sentence processing without segmentation, more data, or a stronger dialect-specific diacritizer.

XI. Figures and Training Graphics

The following figures are embedded from the saved training-output folders. They document the model behavior described above: loss and error trends, learning-rate schedules, sequence accuracy, top-k coverage, and error rates by length. Including the plots in the report is useful because the CSV tables give exact values while the graphics reveal whether training converged smoothly or oscillated. The curves also show why the saved best checkpoints are not always the final epoch; validation sequence accuracy can peak before the last epoch even when training loss continues to improve.

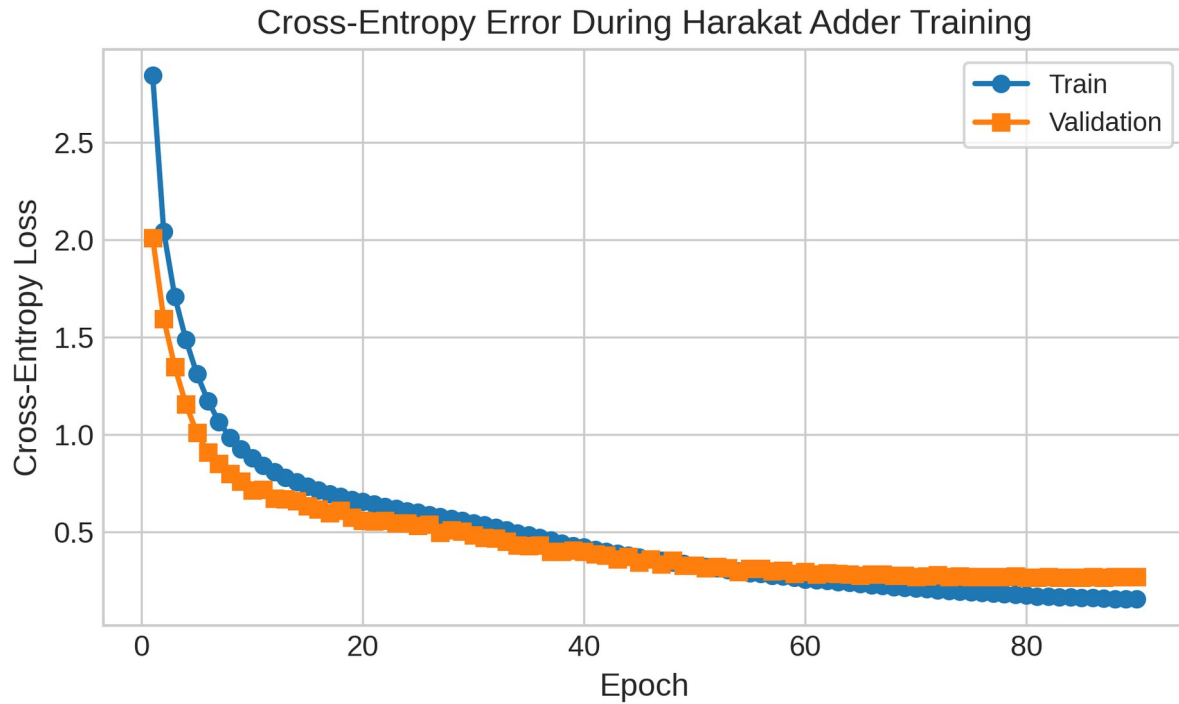


Fig. 1. Harakat-adder training and validation loss/error over epochs.

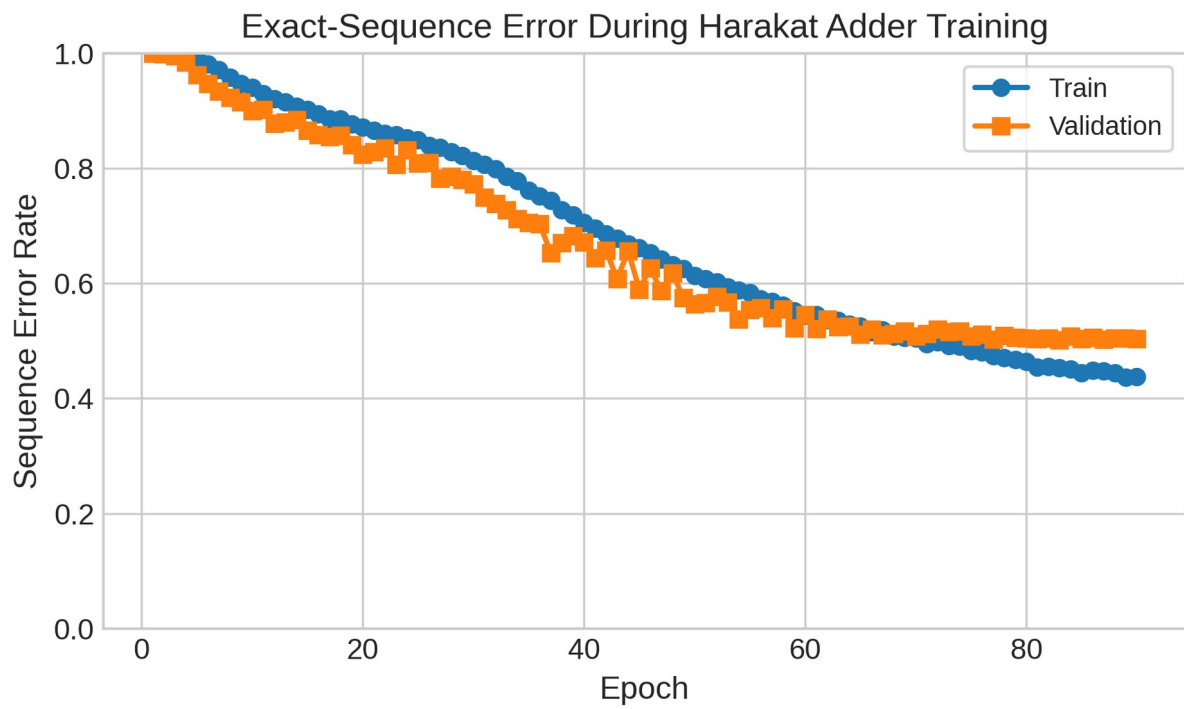


Fig. 2. Harakat-adder sequence accuracy over epochs.

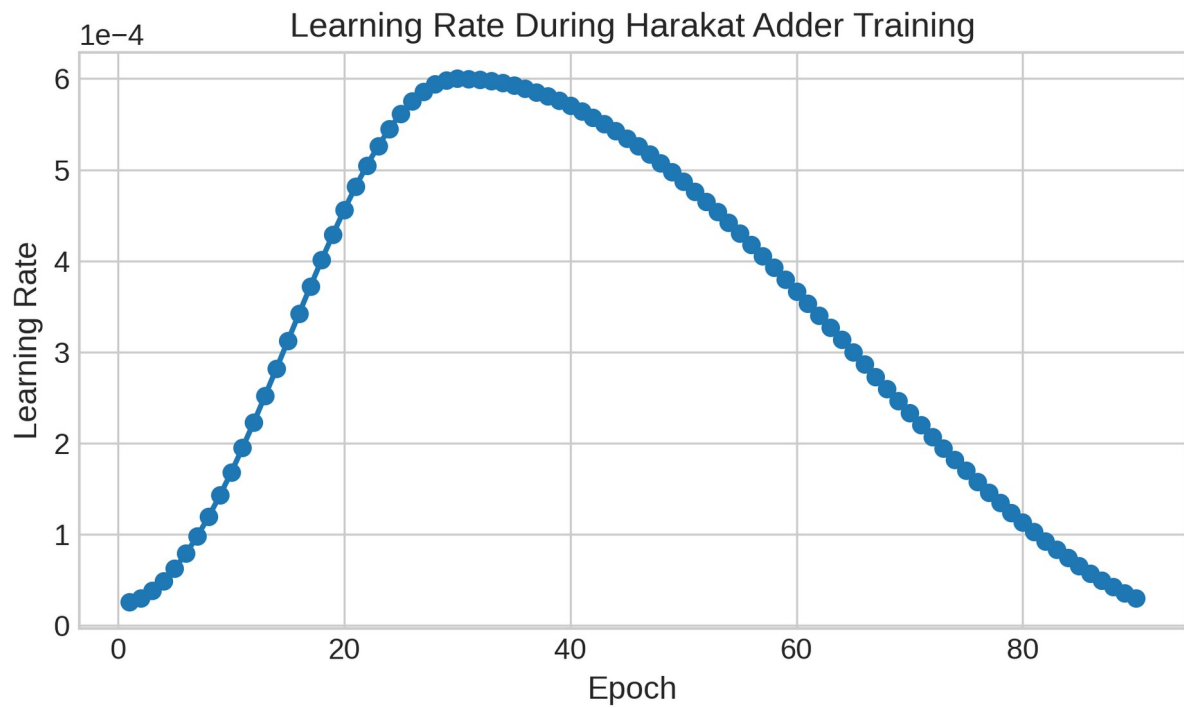


Fig. 3. Harakat-adder learning-rate schedule.

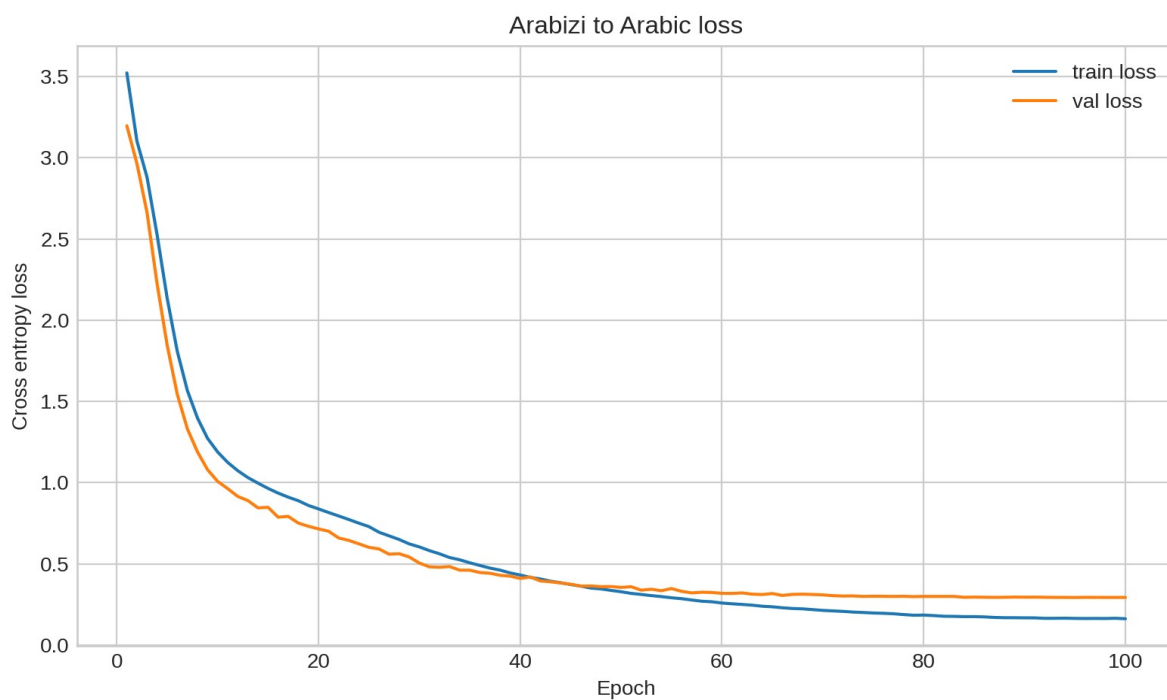


Fig. 4. Arabizi-to-Arabic loss and error trends.

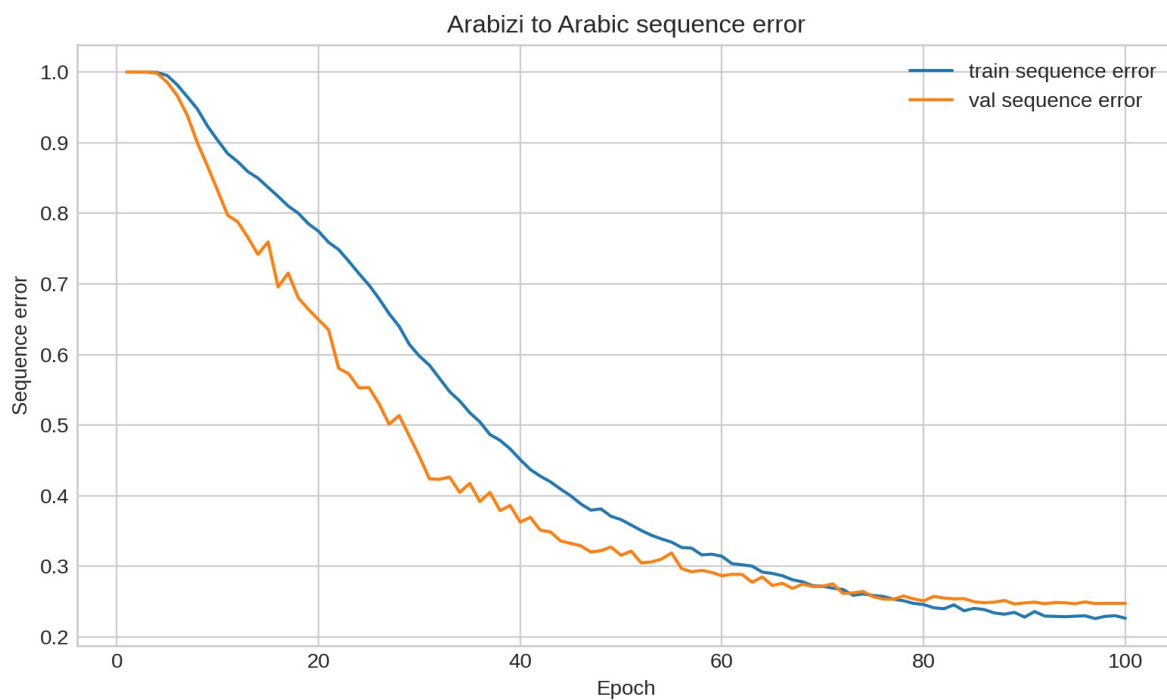


Fig. 5. Arabizi-to-Arabic sequence accuracy trend.

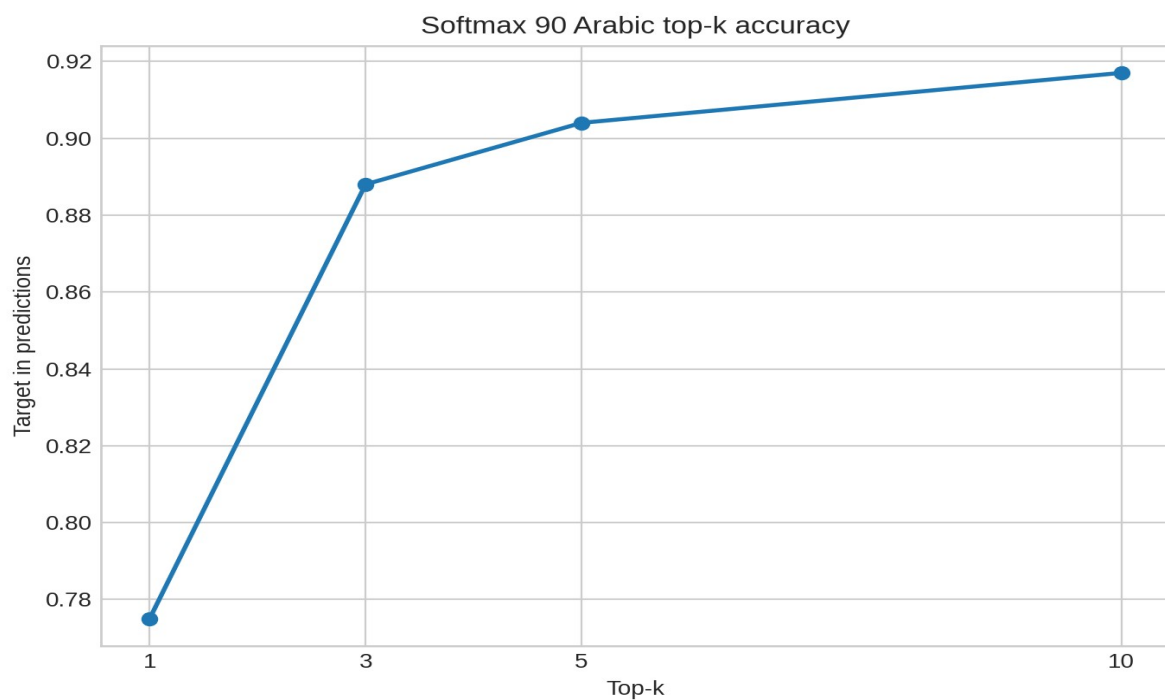


Fig. 6. Arabizi-to-Arabic softmax-90 top-k accuracy.

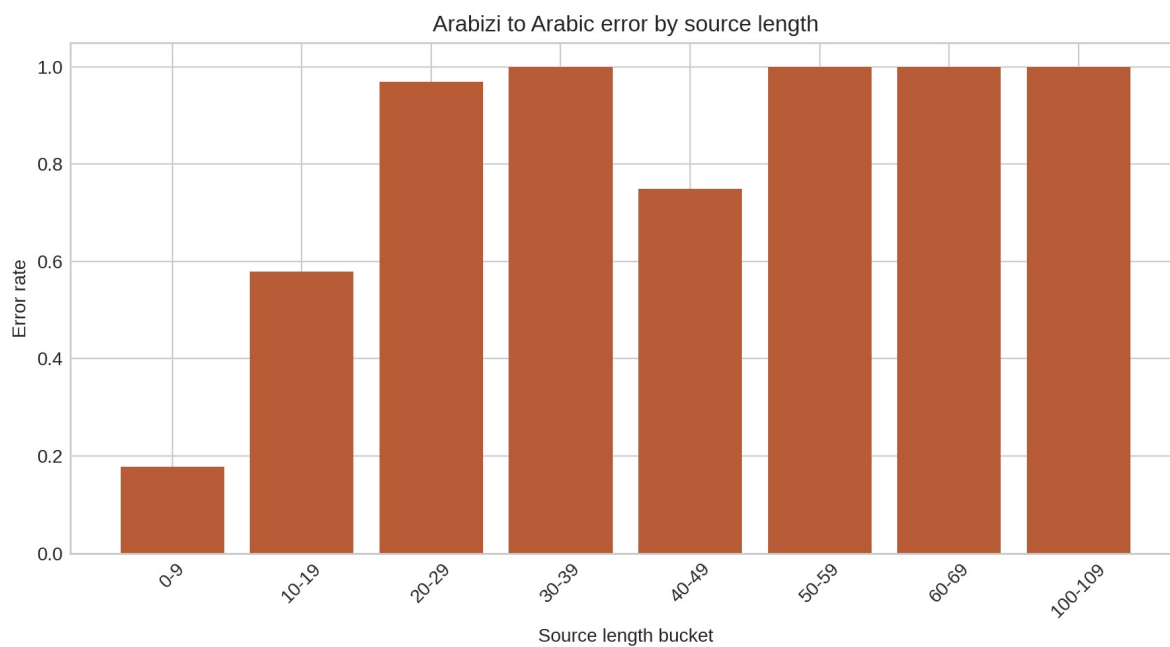


Fig. 7. Arabizi-to-Arabic error rate by source length.

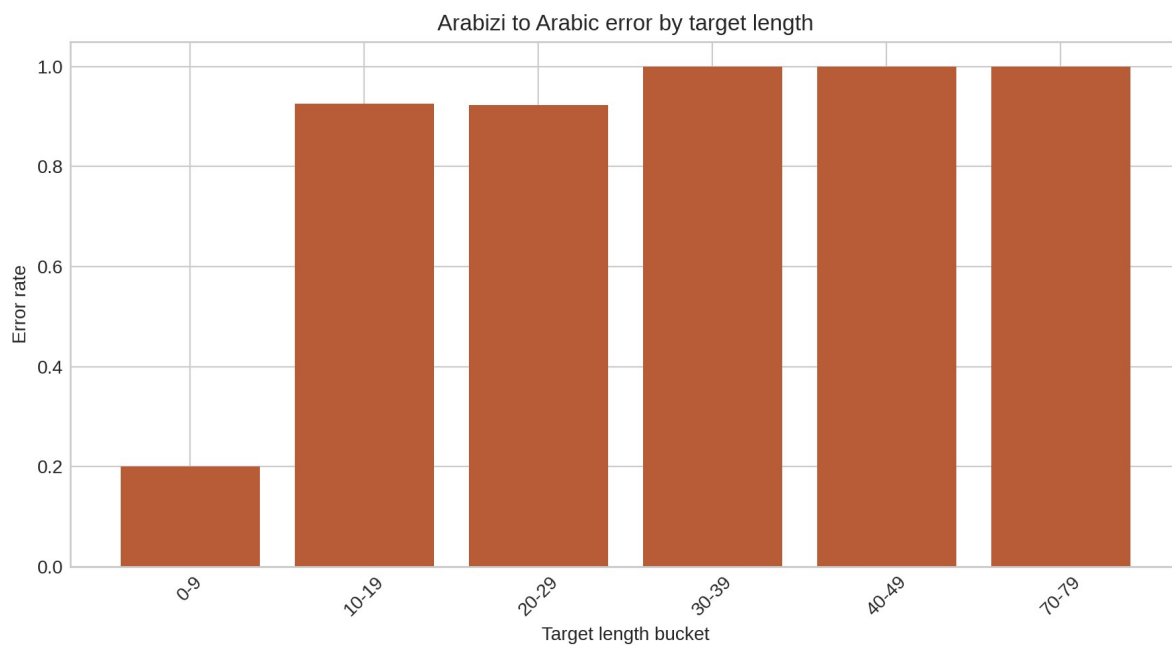


Fig. 8. Arabizi-to-Arabic error rate by target length.

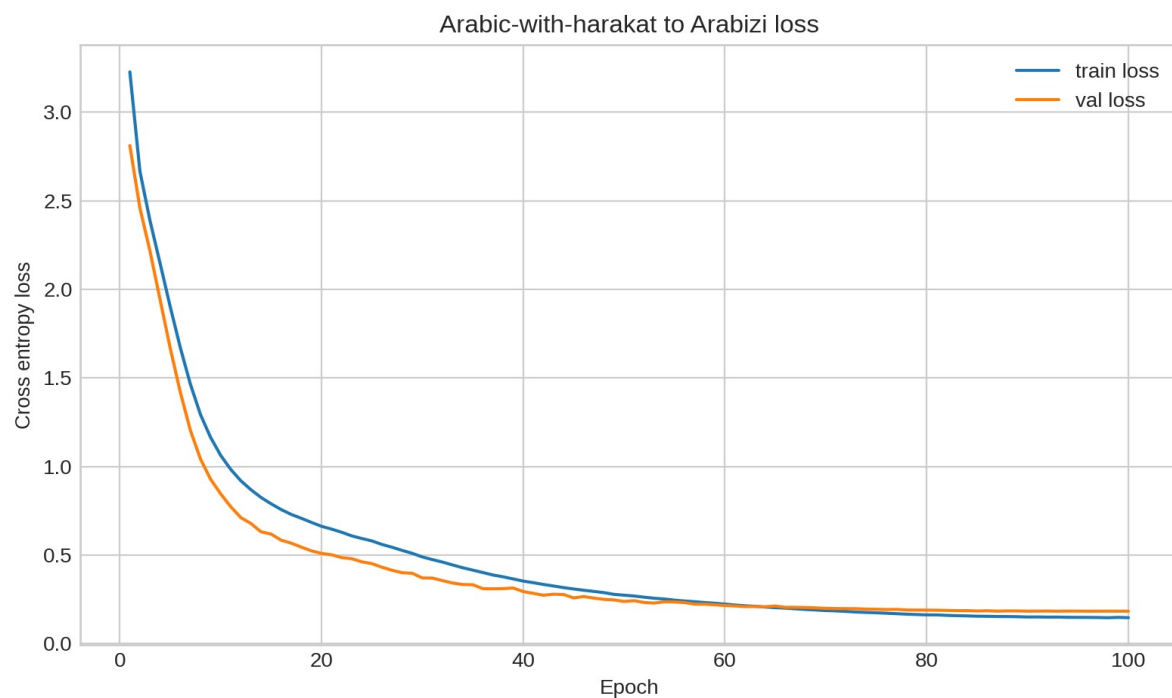


Fig. 9. Original-harakat-to-Arabizi loss and error trends.

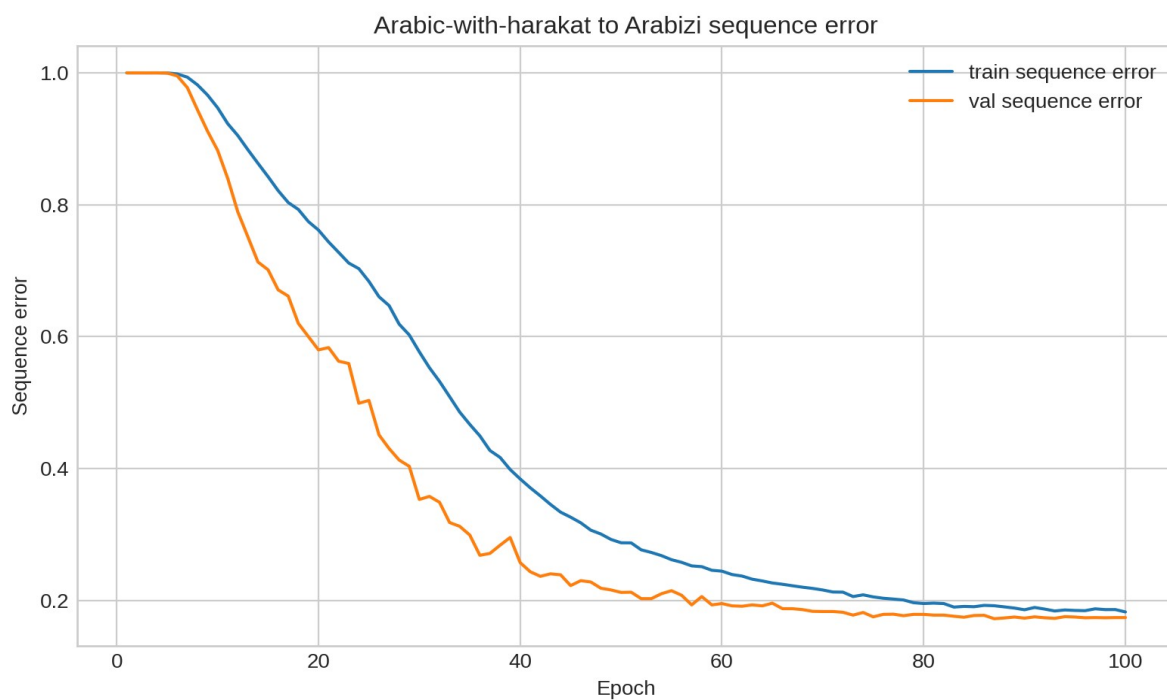


Fig. 10. Original-harakat-to-Arabizi sequence accuracy trend.

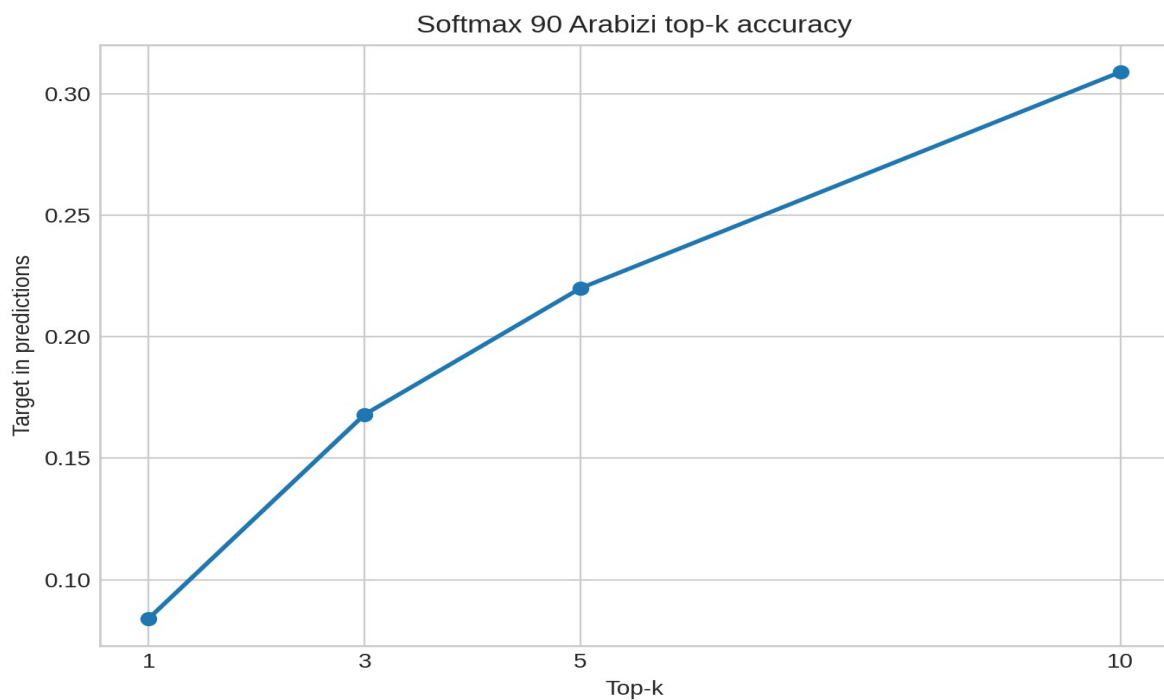


Fig. 11. Original-harakat-trained model softmax-90 stress test on stripped input.

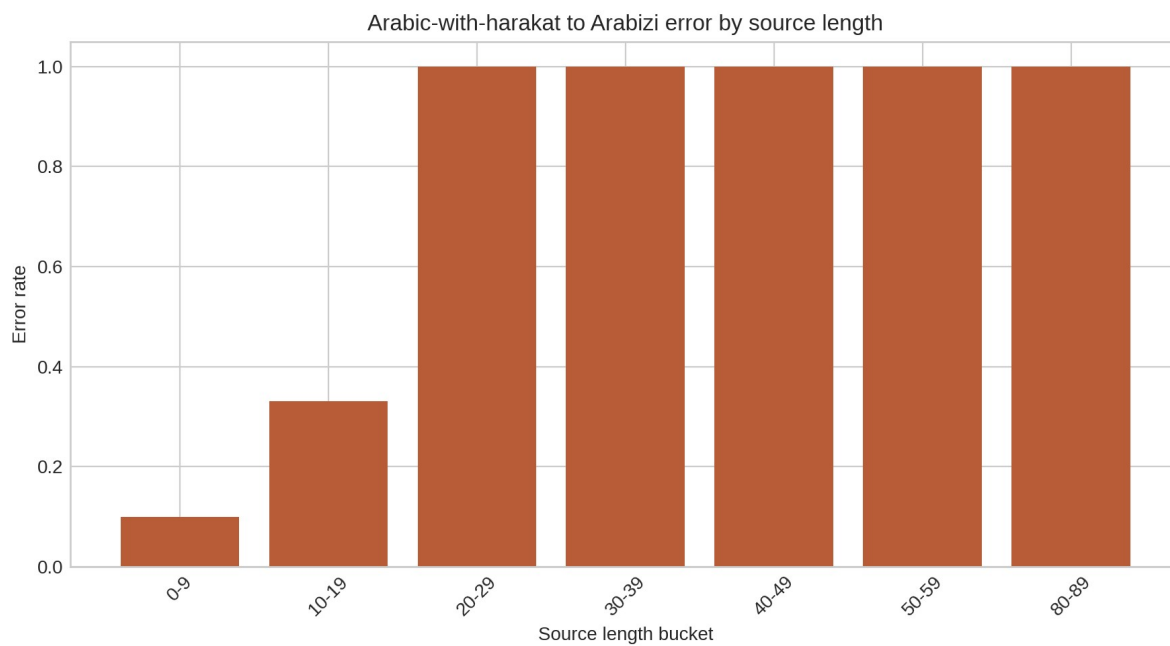


Fig. 12. Original-harakat-to-Arabizi error rate by source length.

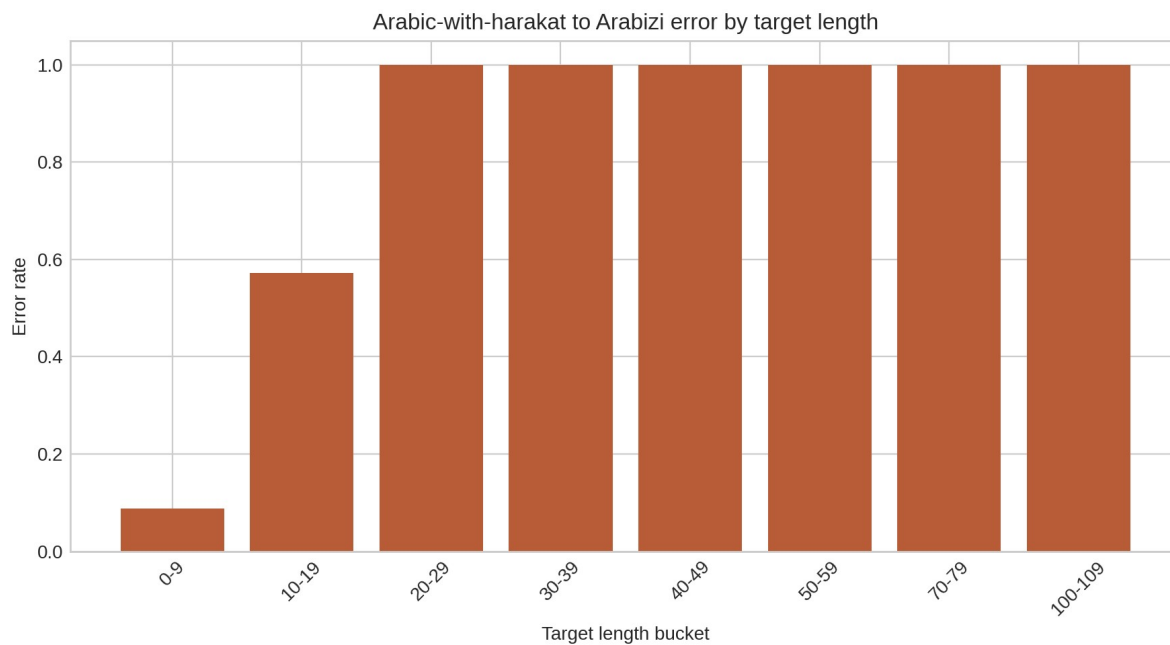


Fig. 13. Original-harakat-to-Arabizi error rate by target length.

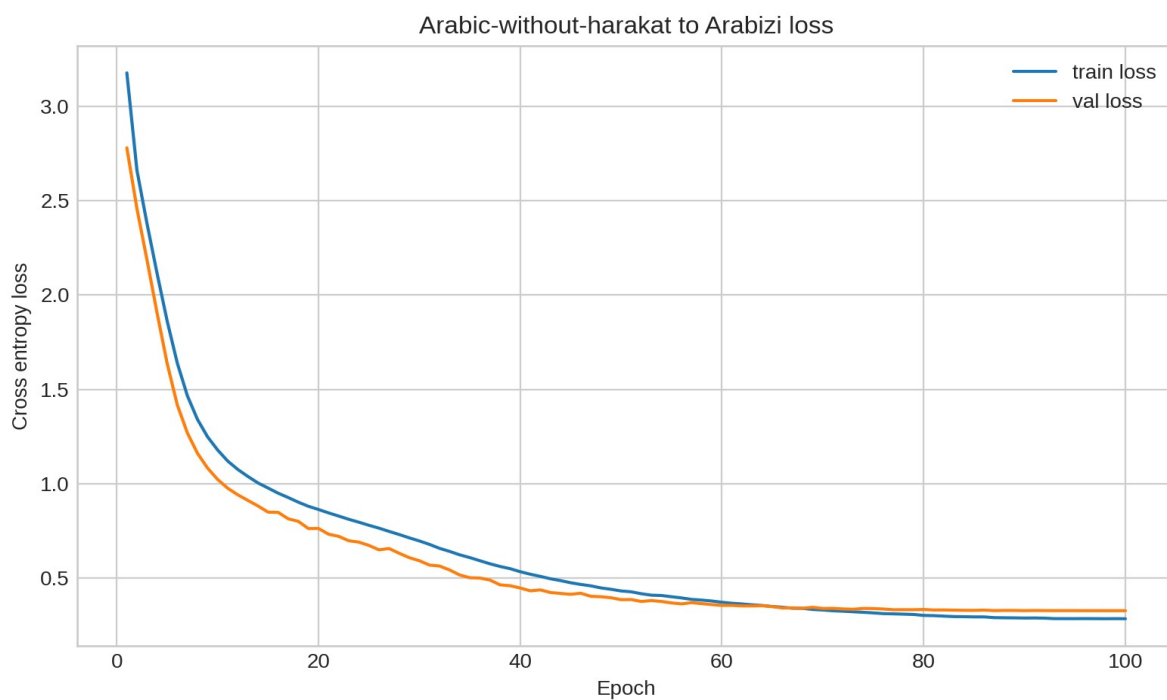


Fig. 14. No-harakat-to-Arabizi loss and error trends.

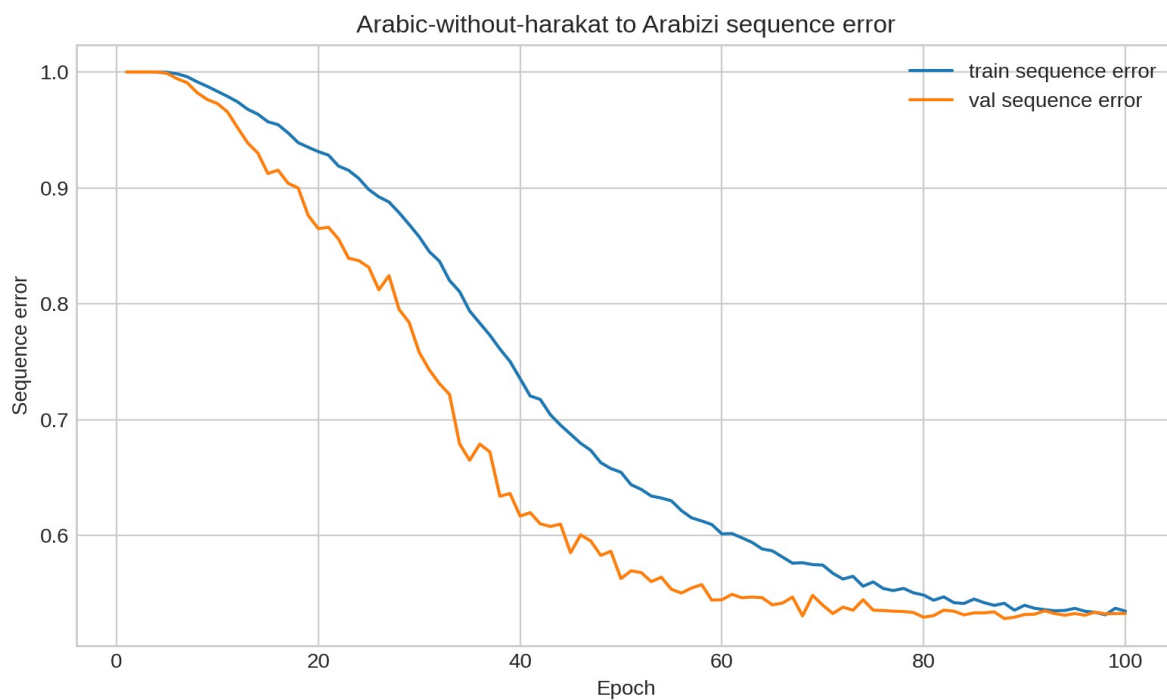


Fig. 15. No-harakat-to-Arabizi sequence accuracy trend.

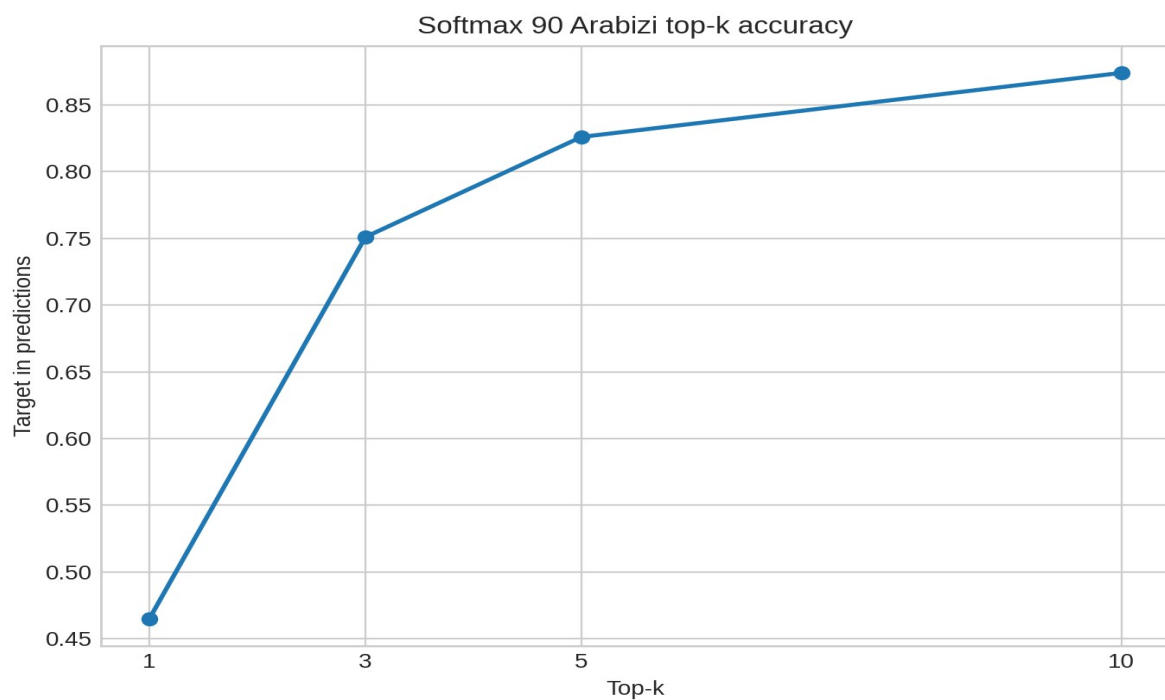


Fig. 16. No-harakat-to-Arabizi softmax-90 top-k accuracy.

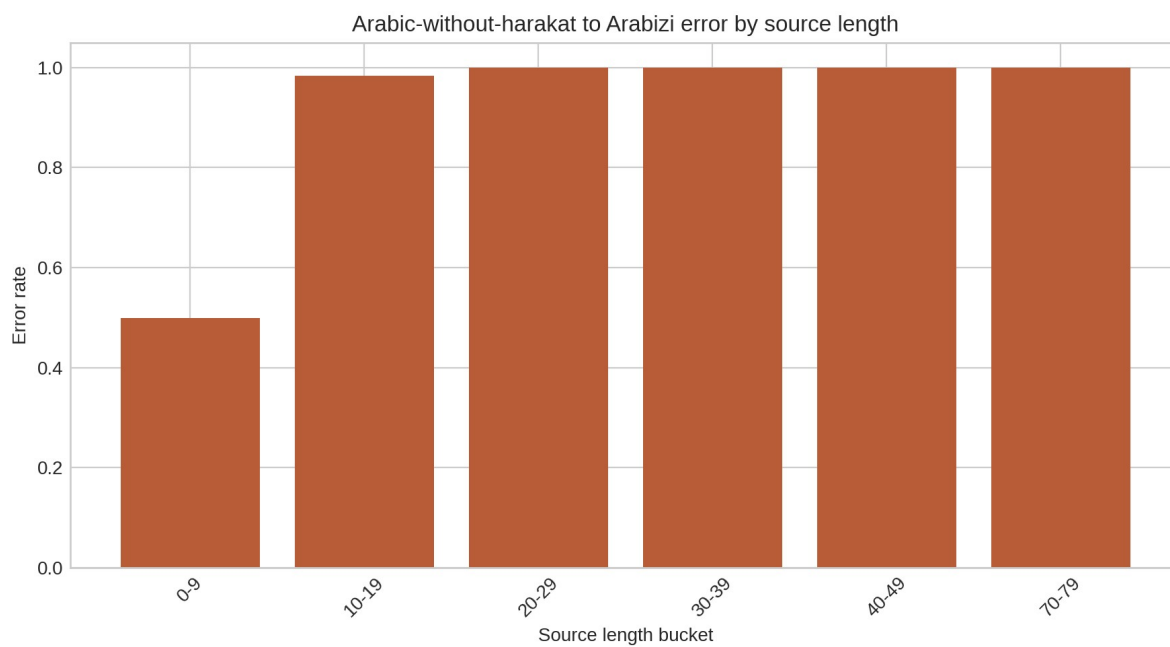


Fig. 17. No-harakat-to-Arabizi error rate by source length.

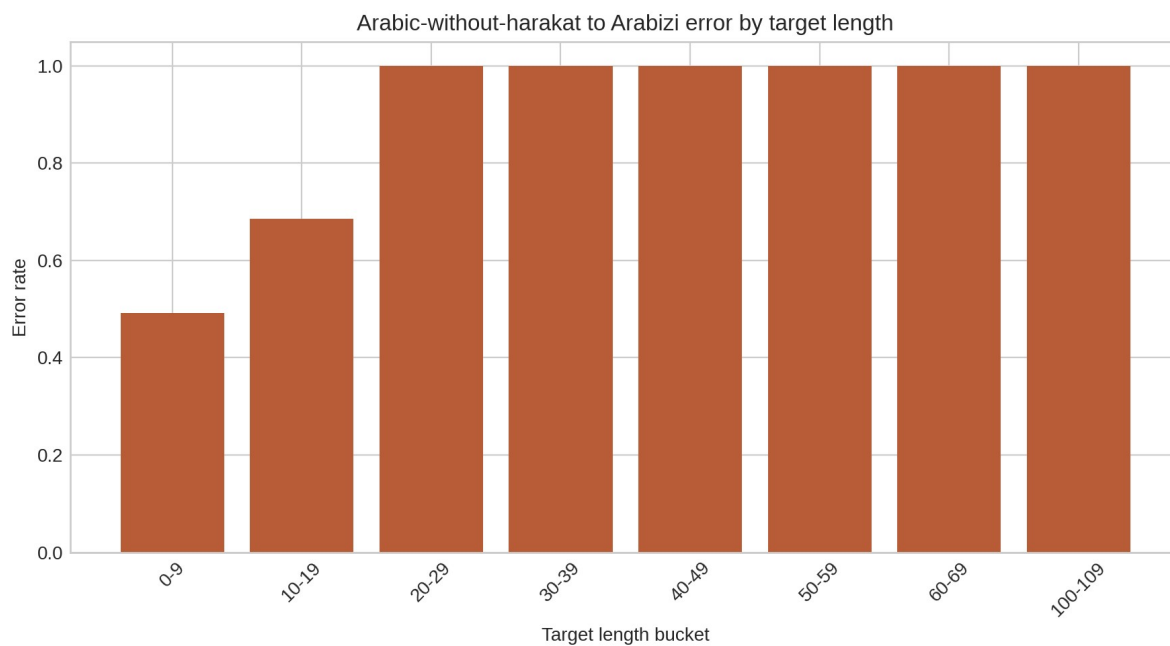


Fig. 18. No-harakat-to-Arabizi error rate by target length.

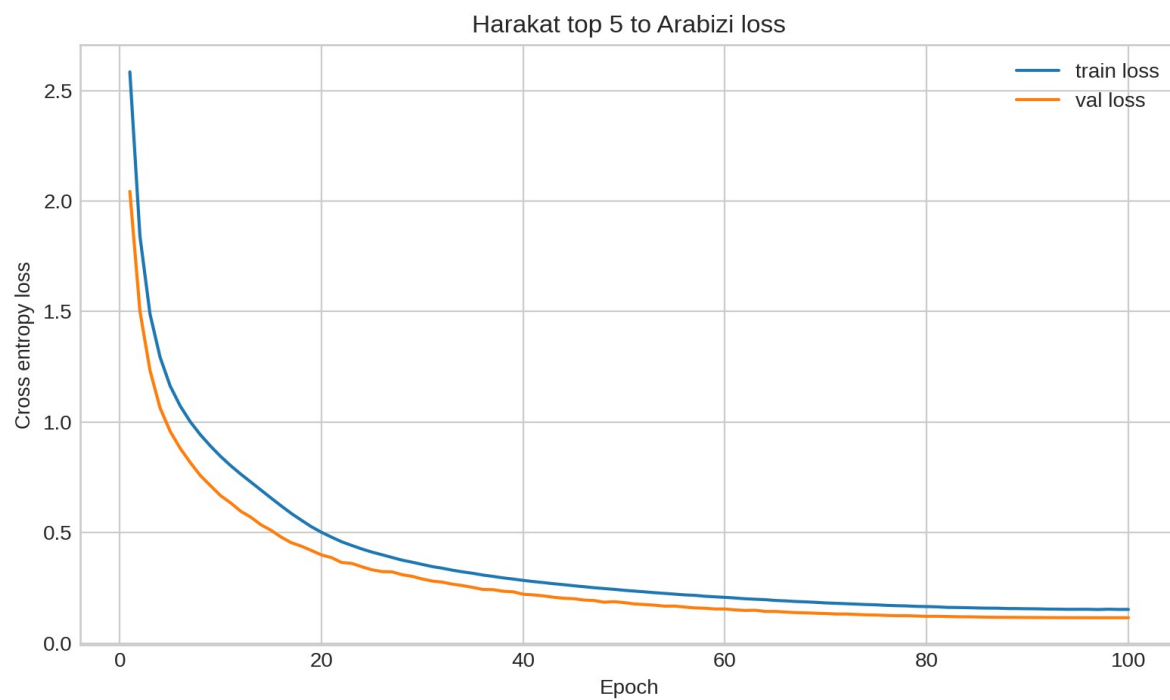


Fig. 19. Harakat-top-5-to-Arabizi loss and error trends.

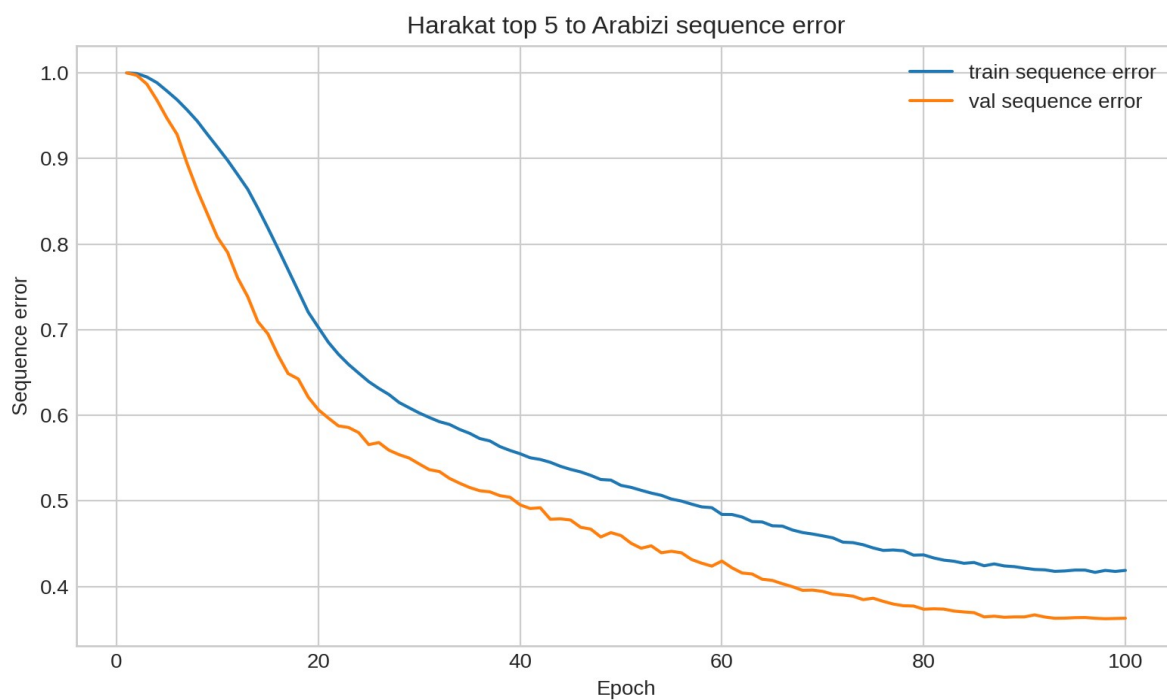


Fig. 20. Harakat-top-5-to-Arabizi sequence accuracy trend.

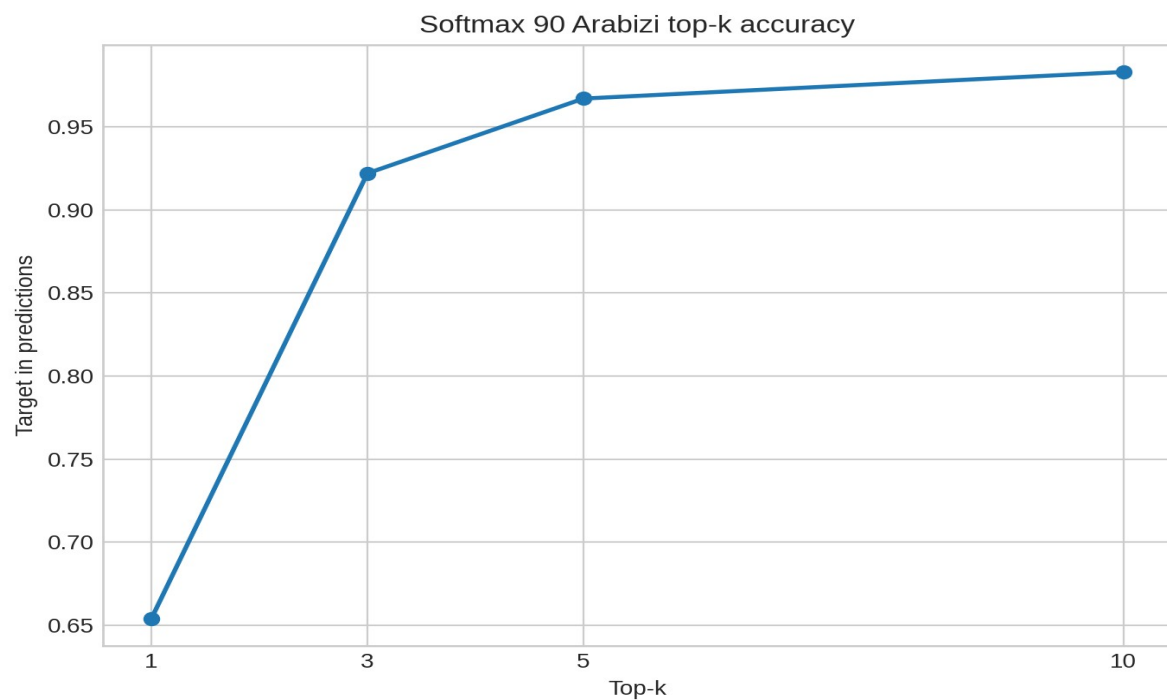


Fig. 21. Harakat-top-5-to-Arabizi softmax-90 top-k accuracy.

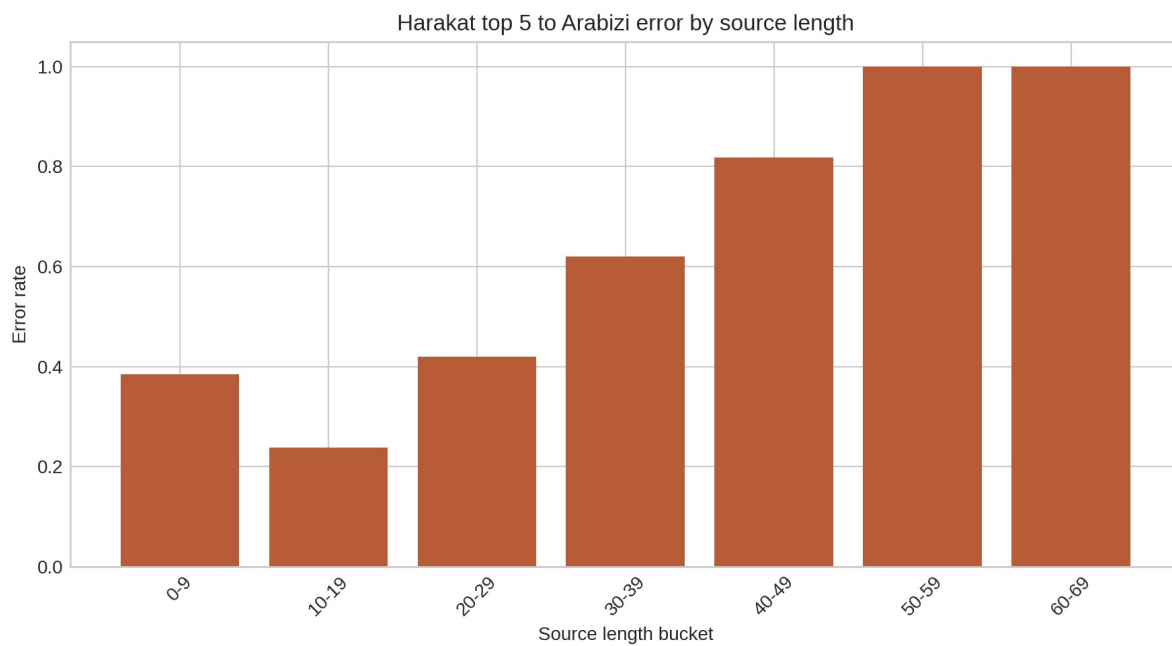


Fig. 22. Harakat-top-5-to-Arabizi error rate by source length.

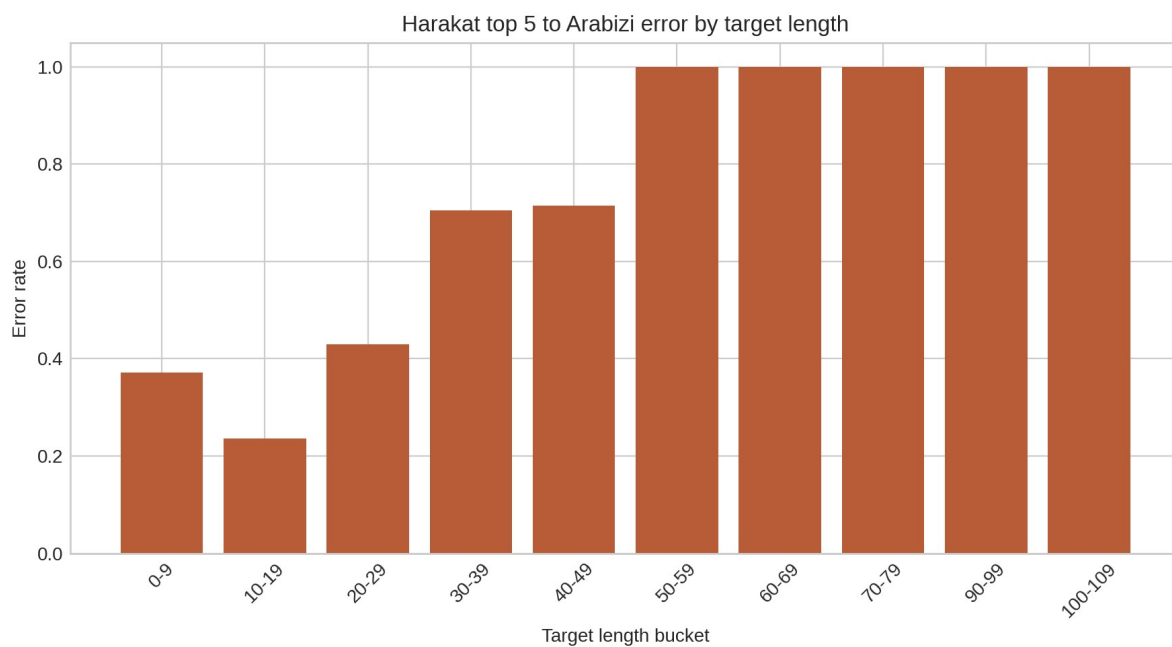


Fig. 23. Harakat-top-5-to-Arabizi error rate by target length.

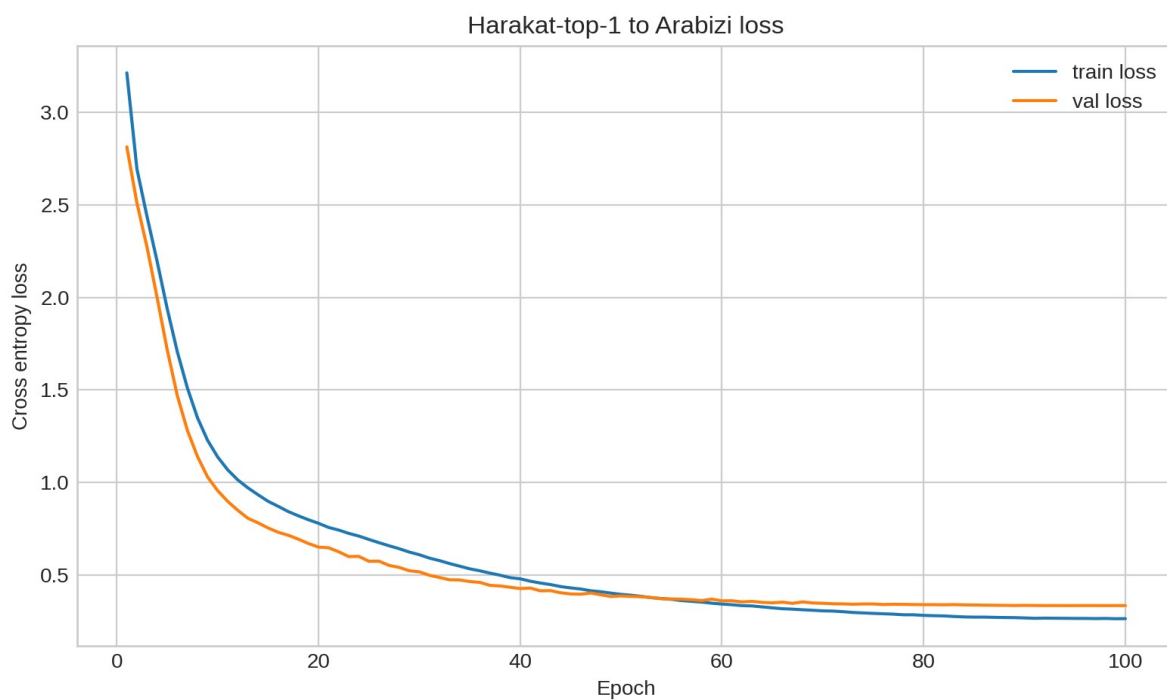


Fig. 24. Harakat-top-1-to-Arabizi loss and error trends.

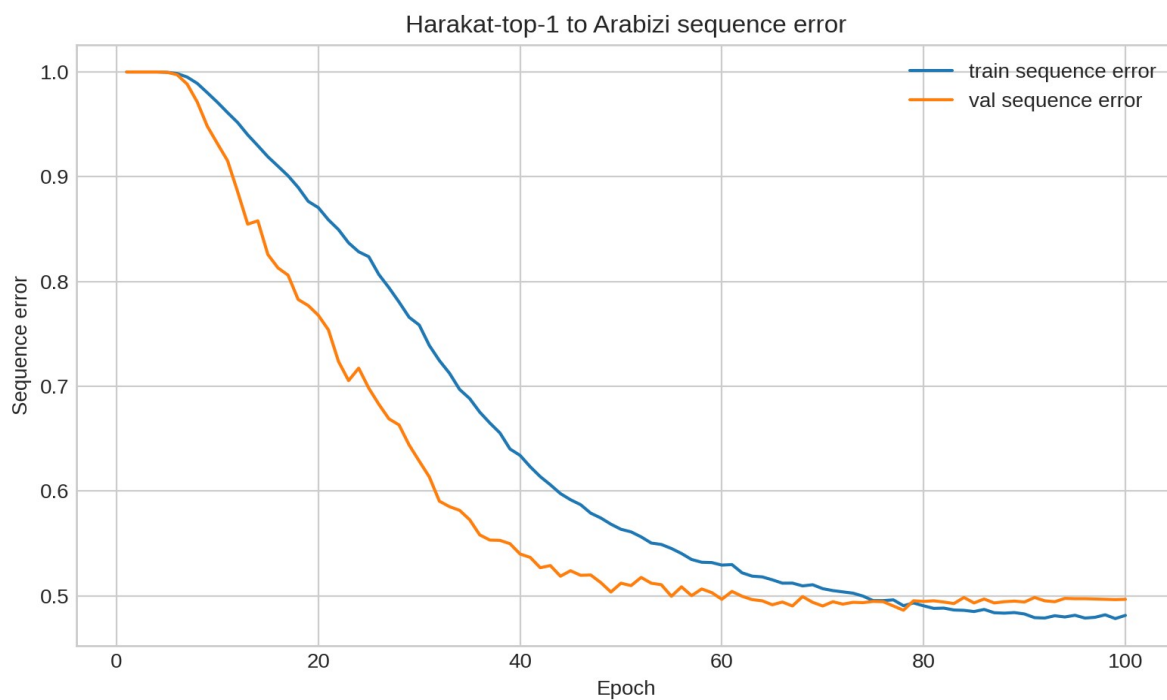


Fig. 25. Harakat-top-1-to-Arabizi sequence accuracy trend.

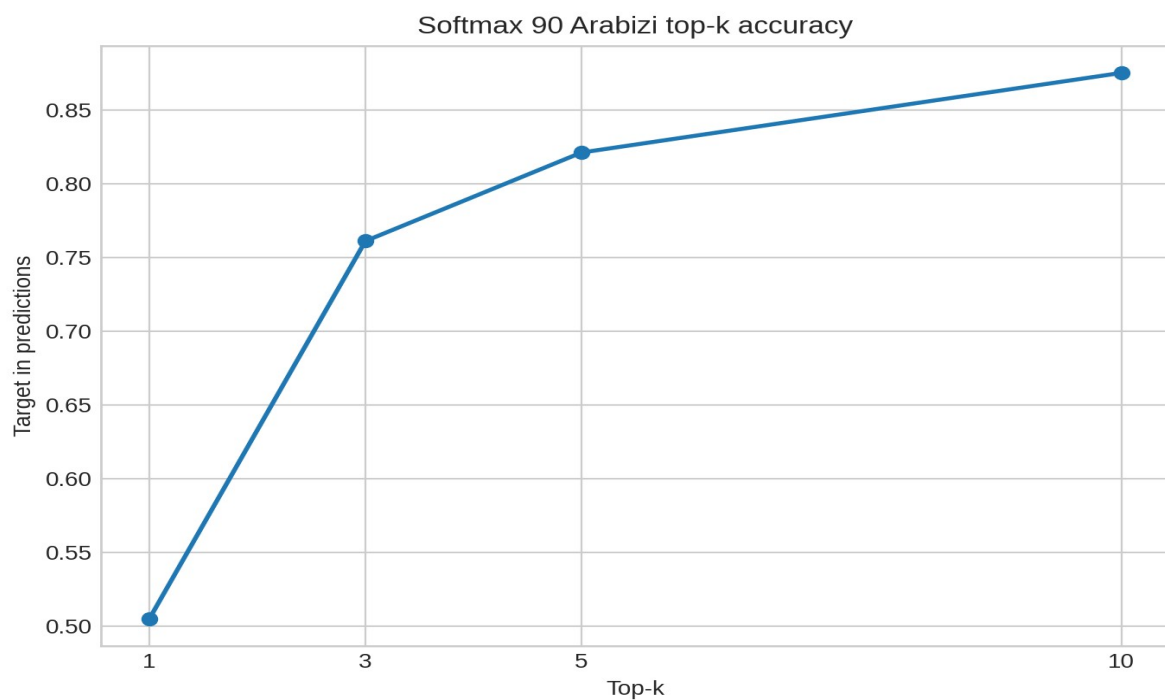


Fig. 26. Harakat-top-1-to-Arabizi softmax-90 top-k accuracy.

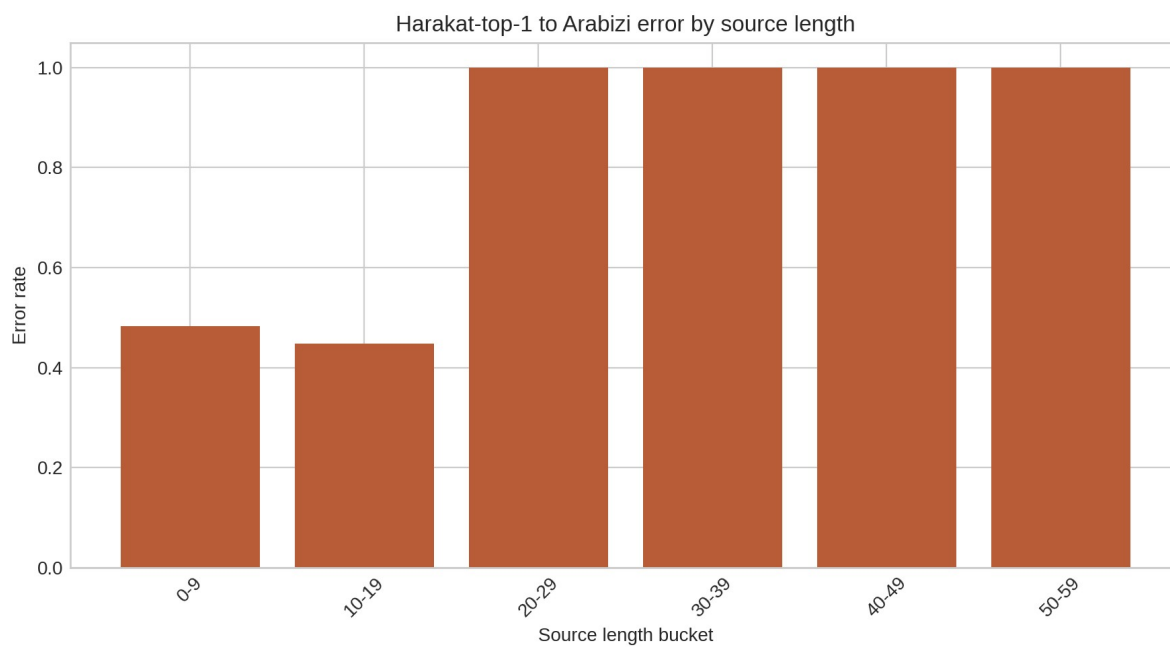


Fig. 27. Harakat-top-1-to-Arabizi error rate by source length.

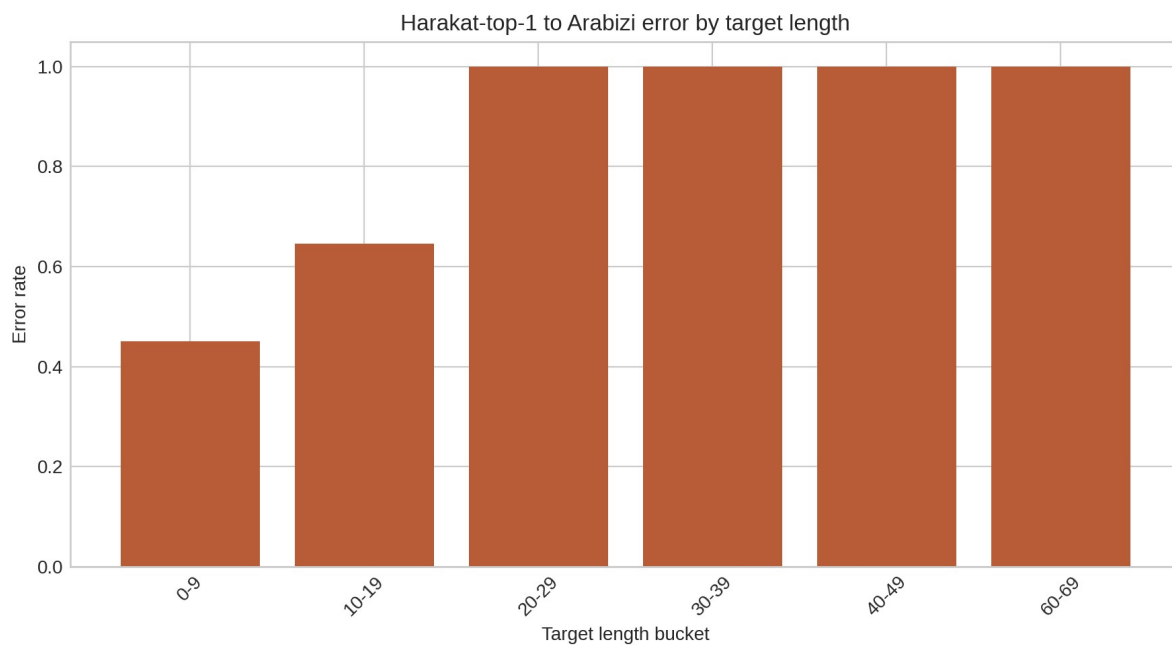


Fig. 28. Harakat-top-1-to-Arabizi error rate by target length.

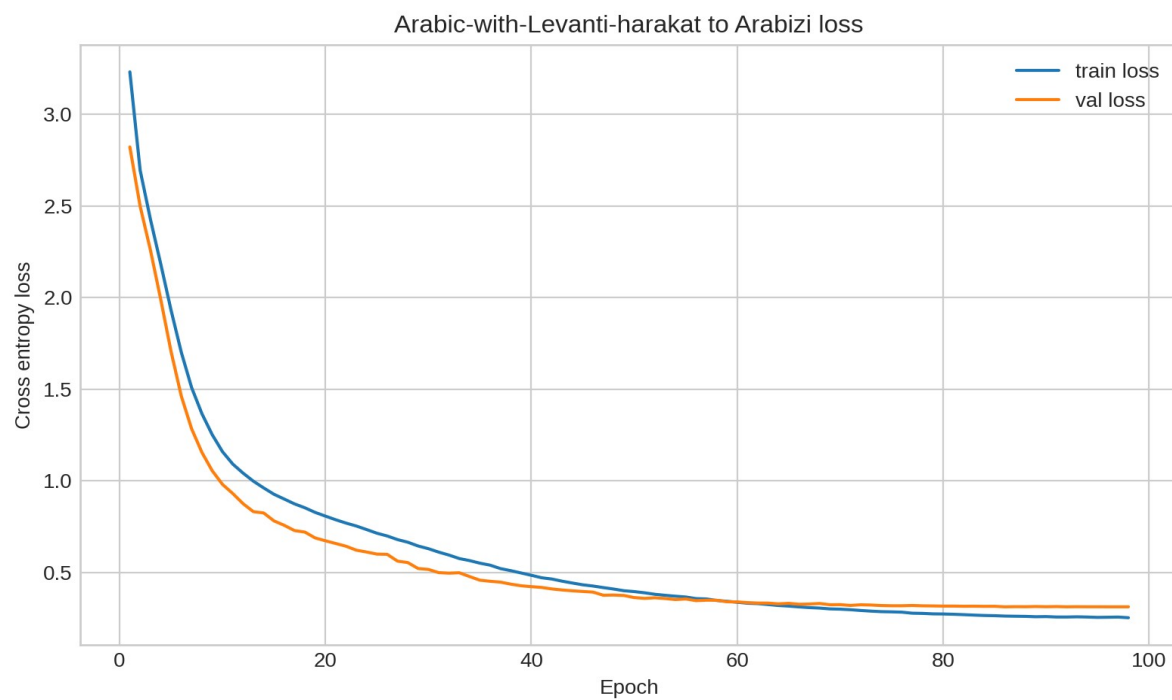


Fig. 29. Levanti-harakat-to-Arabizi loss and error trends.

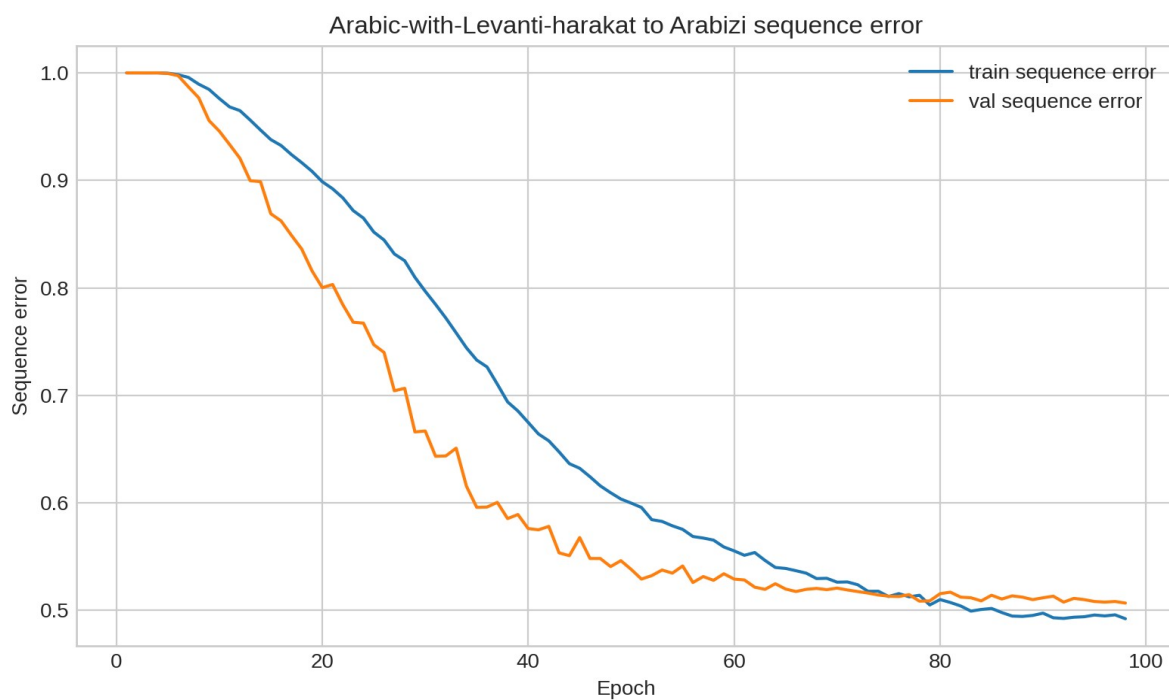


Fig. 30. Levanti-harakat-to-Arabizi sequence accuracy trend.

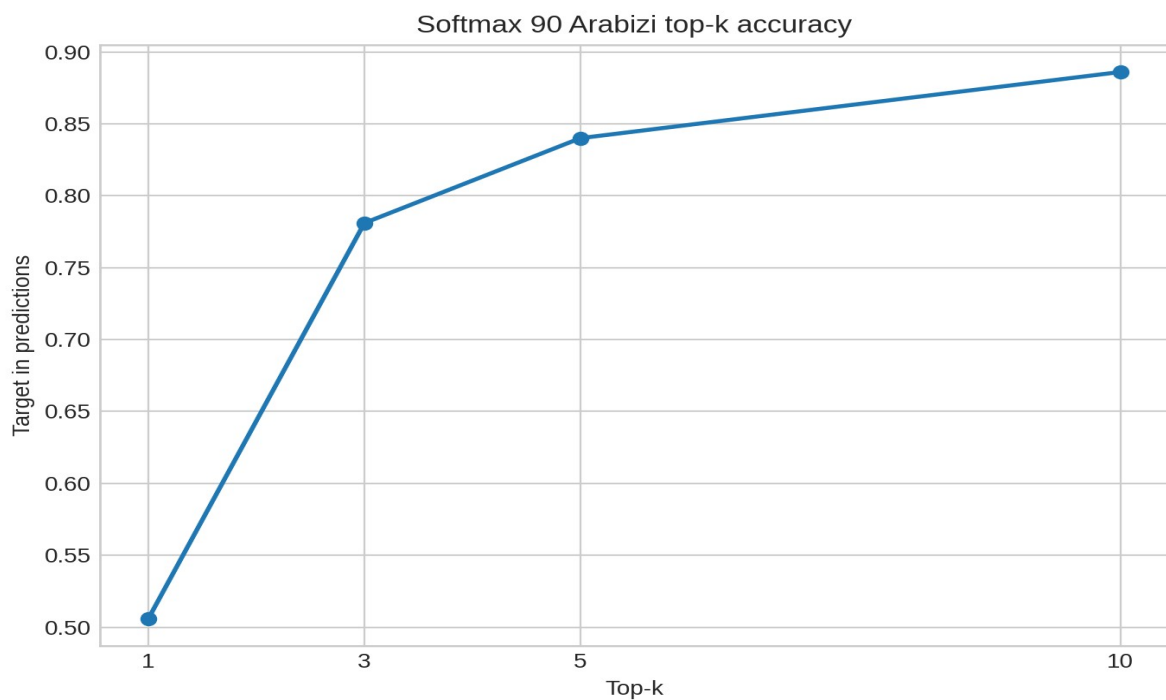


Fig. 31. Levanti-harakat-to-Arabizi softmax-90 top-k accuracy.

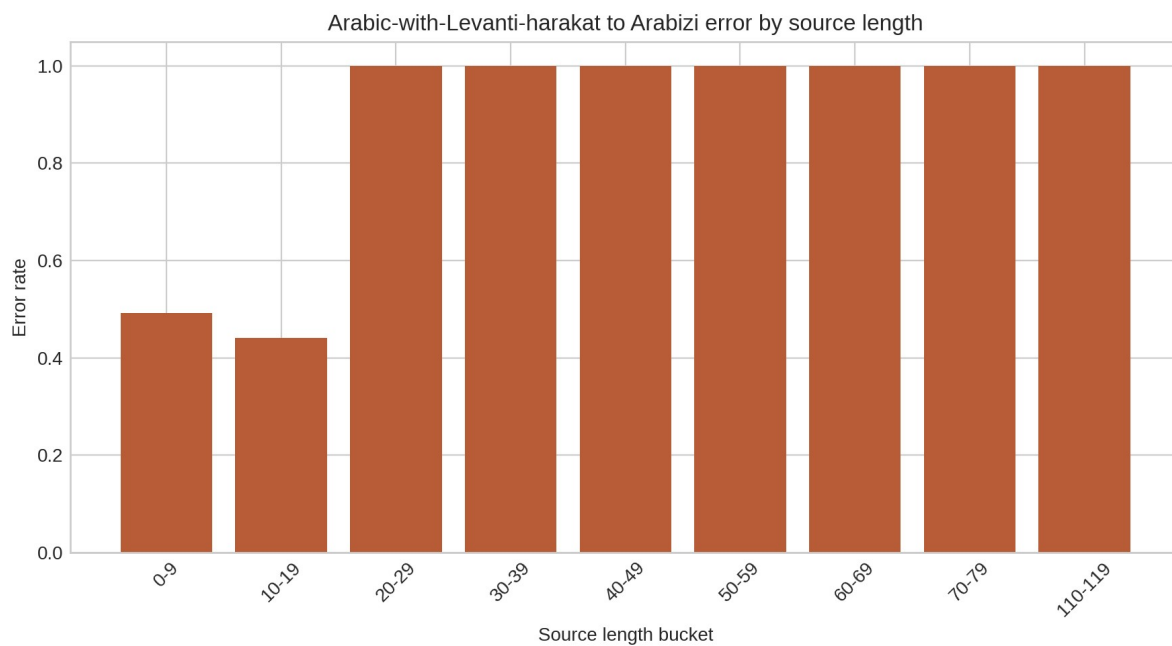


Fig. 32. Levanti-harakat-to-Arabizi error rate by source length.

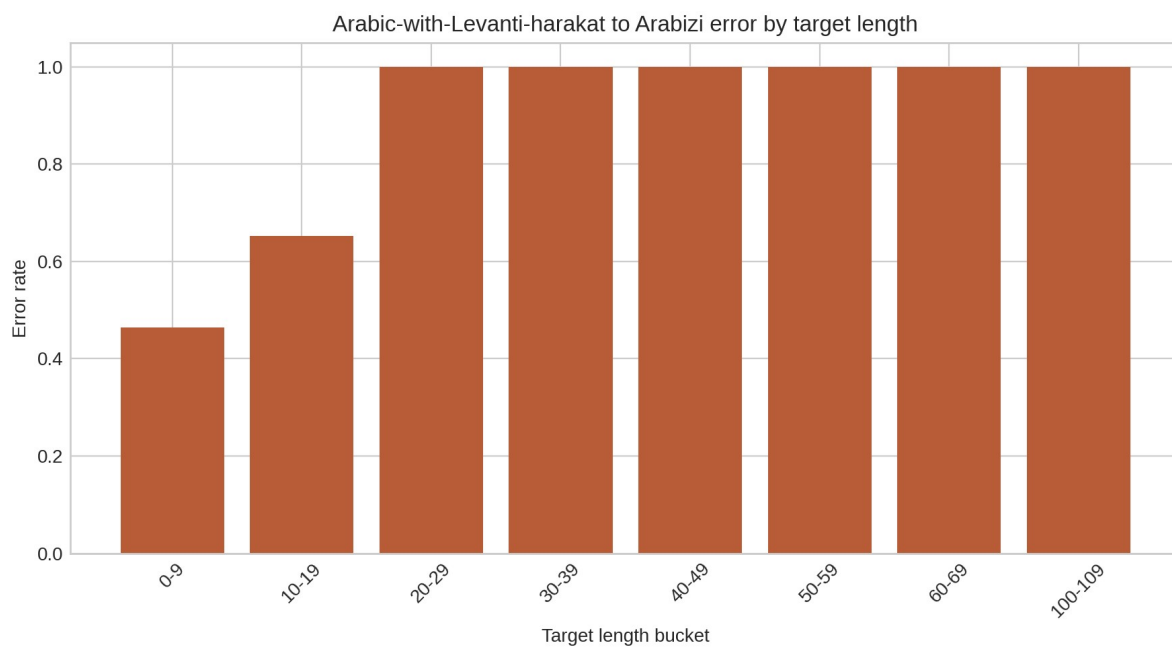


Fig. 33. Levanti-harakat-to-Arabizi error rate by target length.

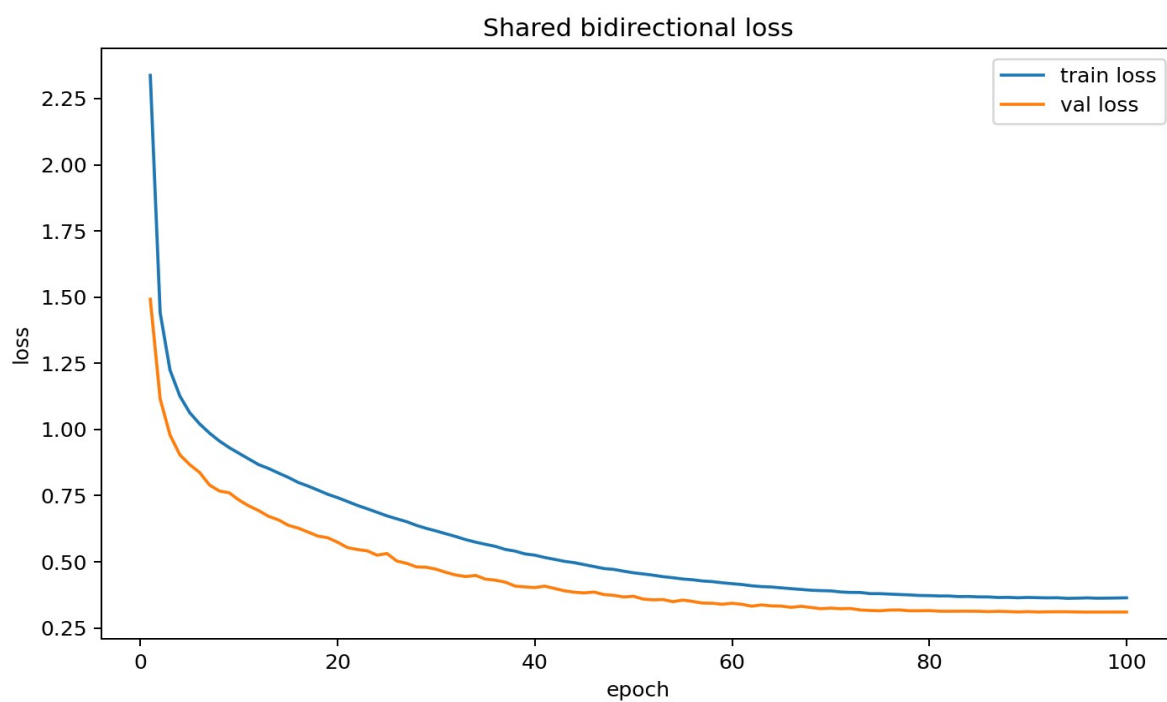


Fig. 34. Shared bidirectional baseline loss trends.

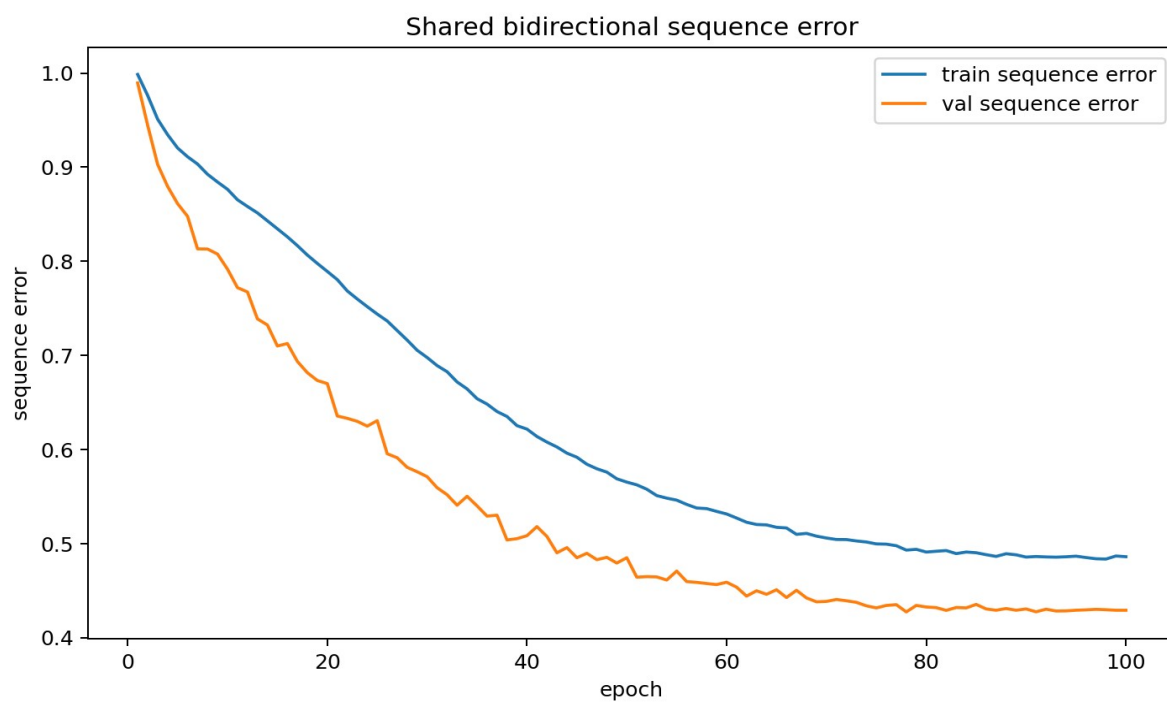


Fig. 35. Shared bidirectional baseline sequence error trends.

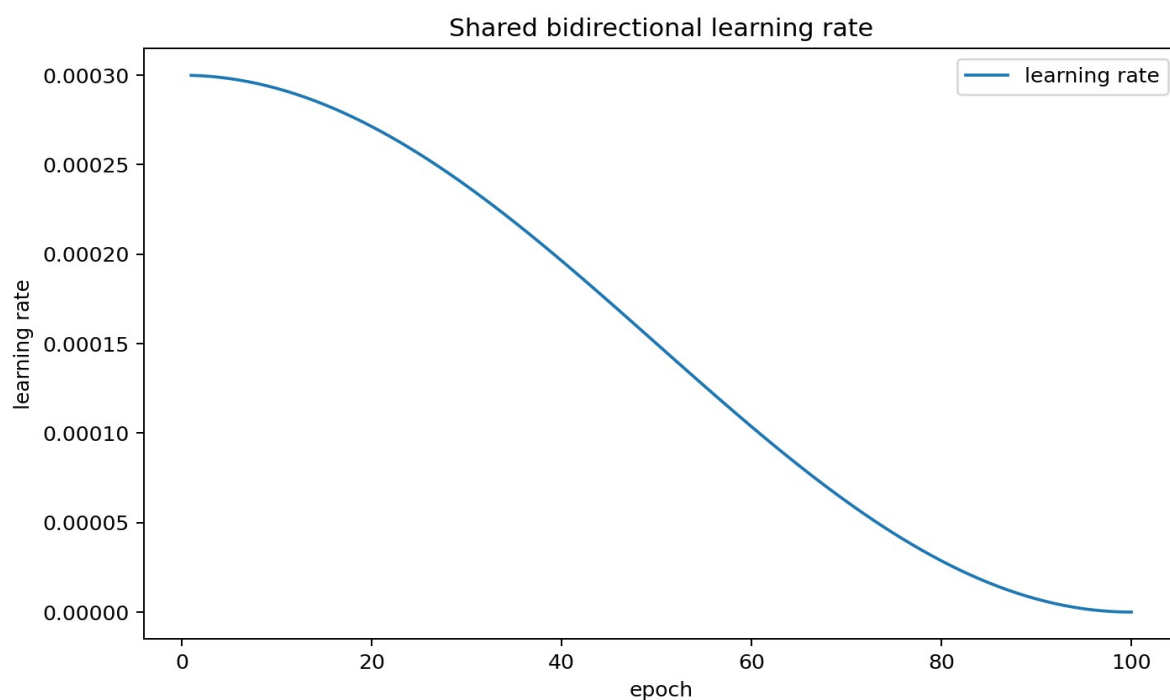


Fig. 36. Shared bidirectional baseline learning-rate schedule.

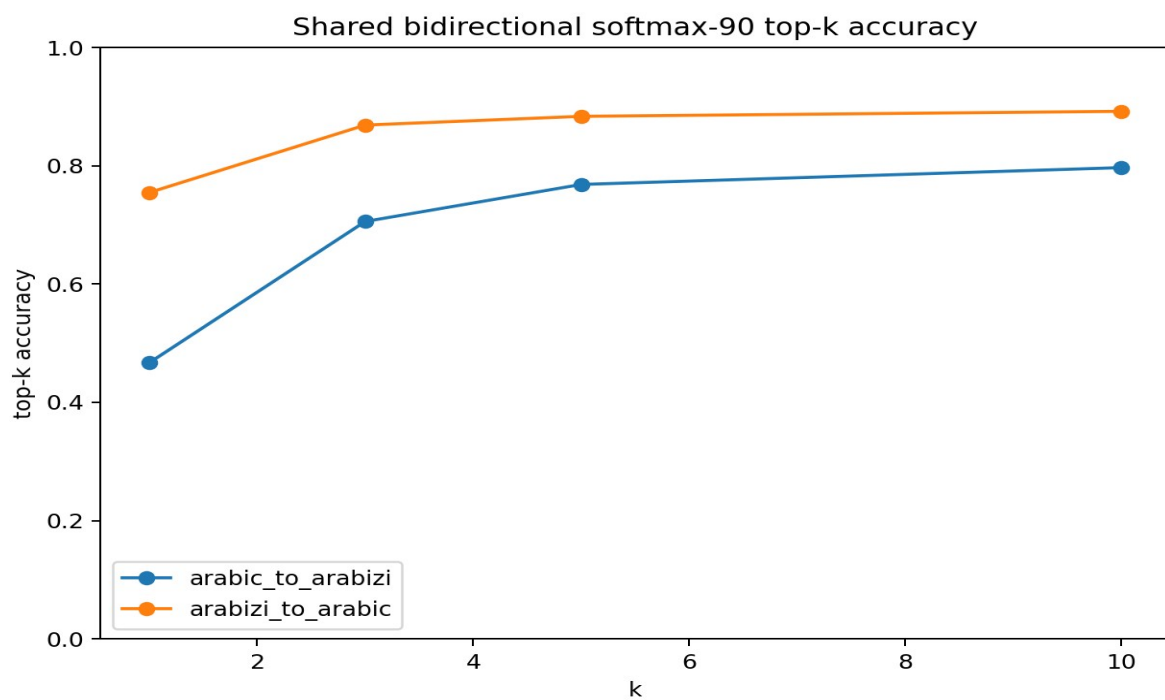


Fig. 37. Shared bidirectional baseline softmax-90 top-k accuracy.

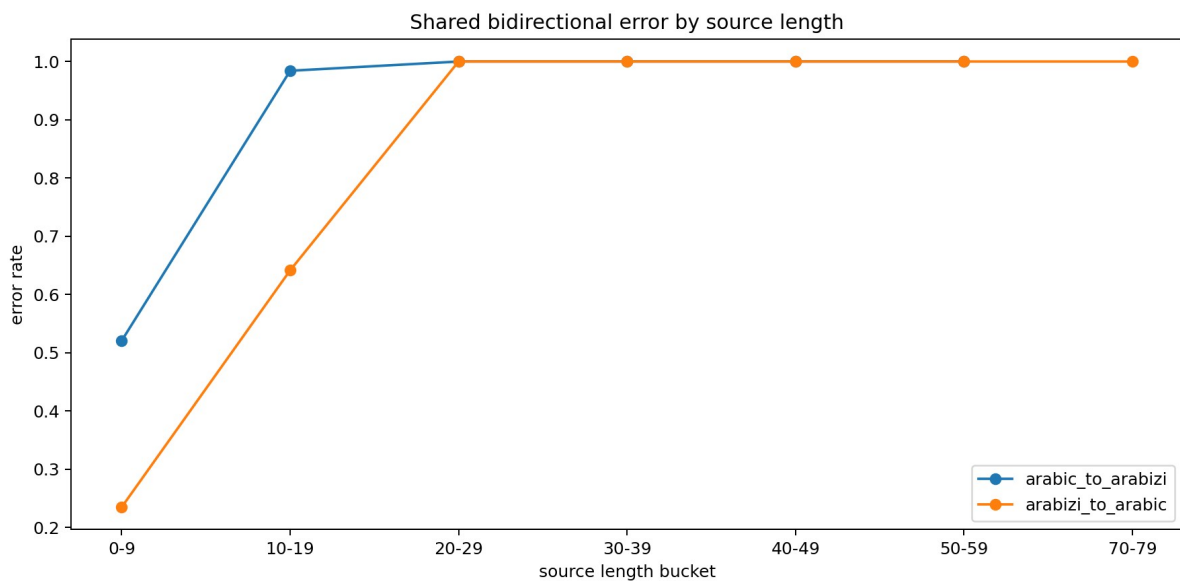


Fig. 38. Shared bidirectional baseline error rate by source length.

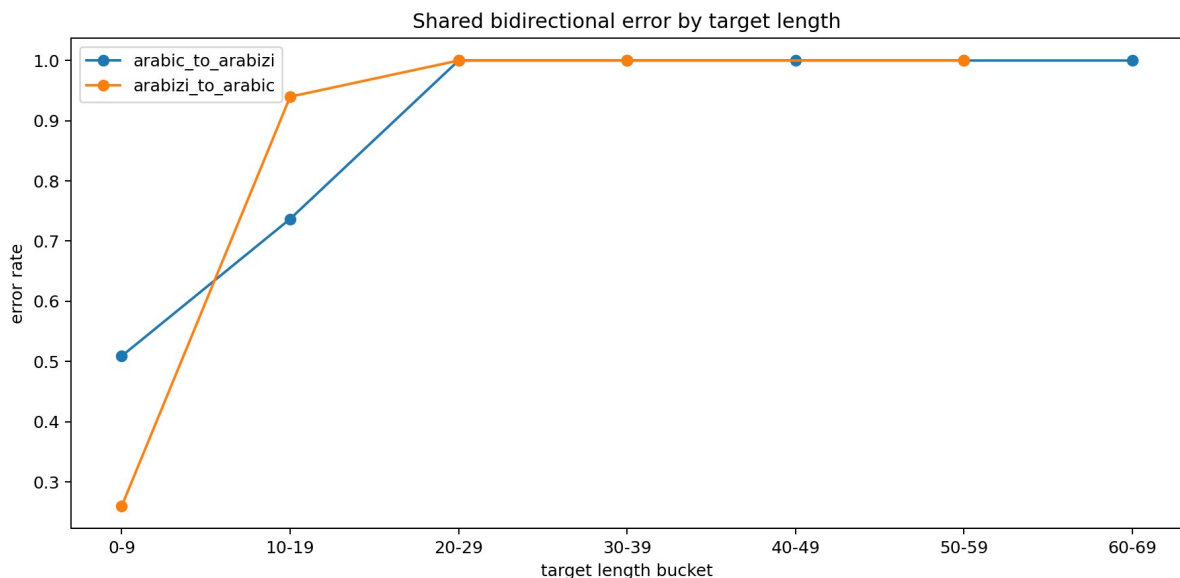


Fig. 39. Shared bidirectional baseline error rate by target length.

XII. Practical Interpretation

The project should be understood as a proof of concept, not as a full-use production app. It demonstrates that a character-level Transformer pipeline can learn useful Palestinian Arabizi/Arabic mappings and that ranked candidate generation is especially valuable in the harder Arabic-to-Arabizi direction. With more time, data, engineering resources, and human evaluation, the same modeling idea could be refined into a stronger tool.

The shared bidirectional baseline is useful mainly as negative evidence. It reached 57.5 percent overall test exact-match accuracy, with 45.1 percent for Arabic-to-Arabizi and 70.0 percent for Arabizi-to-Arabic. Its top-10 coverage was 79.7 percent for Arabic-to-Arabizi and 89.2 percent for Arabizi-to-Arabic. These are respectable results for one shared checkpoint, but they are weaker than the specialized Arabizi-to-Arabic model and much weaker than the selected top-five harakat Arabic-to-Arabizi pipeline.

XIII. Limitations and Future Work

The most important limitation is that this is not ready as an app for full use. It is a proof of concept that demonstrates the feasibility of

the modeling approach and the value of top-k candidate preservation. A full application would require more time, more representative data, a stronger evaluation set, better robustness to noisy inputs, security and privacy review, error monitoring, logging, documentation, and user testing. The current work should therefore be judged as a working research prototype, not as a finished operational system.

The next major limitation is data scope. Maknuune is a high-quality Palestinian Arabic lexicon and the best available dataset identified for this Palestinian Arabizi/harakat task, but it is still primarily lexical. The current models are best for words and short expressions, not long conversational sentences. The length analysis shows why: exact sequence accuracy degrades quickly as the number of characters increases. Future work should add sentence-level data, context-aware decoding, and perhaps a language model for reranking multi-token outputs.

The Arabic-to-Arabizi pipeline also depends on intermediate harakat quality. If the harakat adder fails, the second-stage transliterator receives incomplete evidence. This is the cost of using a pipeline, but it was chosen because ordinary Arabic input lacks the short-vowel information that Arabizi often needs. With more Palestinian Arabic training data, a direct no-harakat model might learn more of that missing information from distributional patterns alone. In the current data setting, the top-five harakat stage is the strongest practical compromise because it gives the Arabizi model multiple plausible pronunciations instead of a single guess. A future version could train the Arabic-to-Arabizi direction end-to-end with latent pronunciation candidates, or jointly train the two stages so that the first stage optimizes downstream Arabizi coverage rather than only diacritized Arabic exact match.

XIV. Conclusion

This project produced a working proof-of-concept model pipeline for Palestinian Arabizi-Arabic transliteration built from cleaned Maknuune data, one custom Arabizi-to-Arabic model, five Arabic-to-Arabizi modeling paths, a shared bidirectional baseline, a custom harakat adder, saved metrics, and plotted diagnostics. It is not a complete app ready for full operational use; it is a feasibility demonstration that shows a promising architecture and a clear path for later refinement with more time, data, validation, and engineering resources. The central argument is that the challenge is not simply moving between two scripts. It is the Arabic-to-Arabizi direction specifically: ordinary Arabic omits many harakat and therefore hides pronunciation cues, while Arabizi itself allows many writer-dependent spellings. Arabizi-to-Arabic reaches 91.7 percent top-10 coverage on the sampled softmax-90 evaluation, but Arabic-to-Arabizi requires explicit uncertainty handling.

The Arabic-to-Arabizi model sequence shows why. The original-harakat model reaches about 83.1 percent exact test accuracy when gold pronunciation marks are present, but that is an upper-bound condition rather than the normal user workflow. The no-harakat model drops to about 47.4 percent exact accuracy, proving that the missing marks matter. The Levanti and custom top-one predicted-harakat models recover some pronunciation information, but both are still limited by one guessed vocalization. The final selected proof-of-concept approach is the custom harakat top-five pipeline because it starts from ordinary Arabic input and performs best operationally by preserving multiple plausible pronunciations before generating Arabizi candidates.

The report also shows why the system is architected as a pipeline rather than one shared model. Arabic without harakat does not fully specify pronunciation, while Arabizi often represents vowels. The shared bidirectional baseline did learn both directions, but it reached only 79.7 percent top-10 coverage for Arabic-to-Arabizi and 89.2 percent for Arabizi-to-Arabic. By contrast, the no-harakat model reaches 87.4 percent top-10 coverage, the Levanti comparison reaches 88.6 percent, the custom top-one comparison reaches 87.5 percent, and the harakat-top-five-to-Arabizi stage reaches 98.3 percent. Those softmax-90 top-k results are the most important practical intelligence result in the paper because they measure whether the right candidate survives into a usable review/search set. The metrics and graphics give clear targets for future deep-learning improvement.

XV. Key Terms

Arabizi. Arabizi is informal Arabic written with Latin letters and numerals instead of Arabic script. For example, writers may use 3 for ع or 7 for ح because those Arabic sounds do not have simple Latin-letter equivalents.

Arabic script. Arabic script is the writing system normally used for Arabic, written from right to left. In this project, Arabic-script forms are compared with Arabizi forms written in Latin characters and numerals.

Harakat / Arabic diacritics. In the Arabic context, harakat are diacritics: small marks above or below letters that indicate short vowels, vowel absence, consonant doubling, and related pronunciation information. This report uses harakat and Arabic diacritics to mean the same thing.

Undiacritized or unvocalized Arabic. Undiacritized Arabic is Arabic script after the harakat/diacritics have been removed. This resembles ordinary Arabic writing, where many short-vowel cues are usually not shown.

Diacritized or vocalized Arabic. Diacritized Arabic is Arabic script with harakat included. In this project, it corresponds to the dataset field that preserves pronunciation marks before they are stripped away.

Diacritization. Diacritization is the task of adding harakat/diacritics back to Arabic text. In this project, the harakat adder performs diacritization before the Arabic-to-Arabizi model generates Romanized candidates.

Transliteration. Transliteration converts text from one writing system to another while preserving approximate pronunciation. Here, the two transliteration directions are Arabic script to Arabizi and Arabizi back to Arabic script.

Romanization. Romanization means writing a language normally written in another script using Latin-script characters. Arabizi is an informal, non-standard Romanized style of Arabic.

Dialectal Arabic. Dialectal Arabic refers to everyday spoken varieties of Arabic, which differ by region in pronunciation, vocabulary, and grammar. This matters because Arabizi spellings often reflect spoken dialect rather than formal written Arabic.

Palestinian Arabic. Palestinian Arabic is the dialect focus of this project. The models are trained on Palestinian-focused data, so their behavior should not be assumed to generalize to all Arabic dialects.

Hamza and alif variation. Hamza and alif are Arabic orthographic elements whose written forms can vary across words and conventions. Small differences in these forms can change exact-match scoring even when outputs remain understandable to a reader.

References

- [1] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," Advances in Neural Information Processing Systems, 2014.
- [2] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," arXiv:1409.0473, 2014.
- [3] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, 2017.
- [4] M. Al-Badrashiny, R. Eskander, N. Habash, and O. Rambow, "Automatic Transliteration of Romanized Dialectal Arabic," Proceedings of CoNLL, pp. 30-38, 2014.
- [5] N. Habash, M. Diab, and O. Rambow, "Conventional Orthography for Dialectal Arabic," Proceedings of LREC, pp. 711-718, 2012.
- [6] O. Hamed and T. Zesch, "A Survey and Comparative Study of Arabic Diacritization Tools," Journal for Language Technology and Computational Linguistics, vol. 32, no. 1, pp. 27-47, 2017.
- [7] A. Fadel, I. Tuffaha, B. Al-Jawarneh, and M. Al-Ayyoub, "Arabic Text Diacritization Using Deep Neural Networks," International Conference on Computer Applications and Information Security, 2019.
- [8] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," Advances in Neural Information Processing Systems, 2019.
- [9] S. Dibas et al., "Maknuune: A Large Open Palestinian Arabic Lexicon," WANLP, 2022, arXiv:2210.12985.

Approximate body word count: 8252